Notation and Symbols

Acronyms

Acronym	Definition
ANOVA	Analysis of Variance
CI	Confidence Interval
CPPN	Compositional Pattern Producing Network
DES-HyperNEAT	Deep Evolvable-Substrate HyperNEAT
EMR	Eager Multi-Resolution (HyperNEAT variant)
ES-HyperNEAT	Evolvable-Substrate HyperNEAT
GEENNS	Grid-based Emergent Evolution of Neocortical Network Substrates
GPU	Graphics Processing Unit
HSHG	Hierarchical Spatial Hash Grid
HyperNEAT	Hypercube-based NEAT
JAX	Just Another XLA (Google's accelerator library)
JIT	Just-In-Time (compilation)
MNIST	Modified National Institute of Standards and Technology (dataset)
MoE	Mixture of Experts
NAS	Neural Architecture Search
NEAT	NeuroEvolution of Augmenting Topologies
SIMT	Single Instruction, Multiple Threads
TOST	Two One-Sided Tests (equivalence testing)
TPE	Tree-structured Parzen Estimator
TRQ	Thesis Research Question
XLA	Accelerated Linear Algebra (compiler)

Mathematical Symbols

Symbol	Definition
θ	Variance threshold governing quadtree subdivision and EMR filtering
D	Depth of quadtree or EMR hierarchical grid
ϕ	Fitness value
ϕ^*	Convergence threshold (default $\phi^* = 0.95$)
w_{min}	Minimum connection weight threshold
N	Population size in the evolutionary algorithm
N_e	Number of experts in the partitioned-experts architecture (Ch. 4)
G	Generation number
G^*	Pruning generation (generation at which early stopping is evaluated)
T^*	Pruning fitness threshold
F_1	F1 classification score (harmonic mean of precision and recall)
d	Cohen's d effect size (standardized mean difference)
r	Rank-biserial correlation (non-parametric effect size)
η^2	Eta-squared (proportion of variance explained)
δ	Cliff's δ (non-parametric effect size for ordinal data)
$\mathcal{C}$	CPPN (pattern-generating network)
$f(\cdot)$	Activation function
f'_i	Adjusted fitness (fitness sharing: $f_i/\lvert S_j\rvert$)
S_j	Species j
n_j	Offspring allocation for species j
δ_t	Compatibility threshold (NEAT speciation)
τ	Survival threshold (truncation fraction)
P	Number of GPU cores / parallel units
$\mathbf{x}$	Vector (bold lowercase)
$\mathbf{W}$	Matrix (bold uppercase); substrate weight matrix when subscripted
$\mathcal{S}$	Set (calligraphic)
$\mathbb{1}[\cdot]$	Indicator function (1 if condition is true, 0 otherwise)

Conventions

- Vectors are denoted by bold lowercase letters ($\mathbf{x}$), matrices by bold uppercase ($\mathbf{W}$), and sets by calligraphic letters ($\mathcal{S}$).
- Statistical significance is reported at the $\alpha = 0.05$ level unless otherwise stated.
- Confidence intervals are 95% bootstrap intervals with 10,000 resamples unless otherwise noted.
- Effect sizes follow Cohen's conventions: small ($d = 0.2$), medium ($d = 0.5$), large ($d = 0.8$).

- All experimental results use median as the measure of central tendency (robust to outliers in evolutionary fitness distributions).
- Statistical results follow a standardized reporting format: success rate as percentage with sample size, central tendency as median, uncertainty as 95% bootstrap confidence interval (10,000 resamples), effect size as Cohen's d or rank-biserial r, and significance via the appropriate non-parametric test (Mann-Whitney U for two-group comparisons, Kruskal-Wallis for multi-group designs).

Chapter 7

The GEENNS Vision: Proof of Concept and Future Directions

> Your mind is for having ideas, not holding them.
>
> David Allen

This chapter serves two purposes: it frames the thesis within the broader GEENNS research program, and it reports proof-of-concept prototype evidence that validates the compositional pipeline this thesis enables. We present the biological foundations and algorithm architecture that motivate GEENNS, identify three barriers that Chapters 3–6 remove or enable solutions to, and close with an honest assessment of what remains open. The prototype demonstrates that frozen specialist grids composed through evolved mappers work at small scale and maintain 100% success up to 5-grid Boolean scale (and a 6-grid probe where the augmented pipeline first falters and gated routing recovers it), where every monolithic baseline collapses to 0%; this finding is based on approximately 3,100 runs across Boolean compounds and continuous additive compounds (through eight gates). The best individual mapper does not zero-shot unseen compositions (in the compositional generalization test, best-case fitness is 0.53–0.80, population mean near 0.53), yet seeding new searches with populations evolved on related compositions (warm-start transfer) reveals the compositional signal is present at the population level in the form of rare latent solvers. Compositional generalization is therefore not answered in the negative; it is refined from "does it generalize?" into "how do we extract the population-level signal into a single mapper?" The infrastructure built in earlier chapters now makes that question approachable. This chapter delivers a proof of concept and charts the path for future work. The formal mathematical specification appears in Appendix B.

7.1 Motivating Hypotheses and Scope

GEENNS draws on three disciplines: distributed cognition from psychology, where human problem-solving is collaborative across specialized faculties [42]; the cortical column hypothesis from neuroscience, where Mountcastle [69] identified a repeated canonical microcircuit that produces diverse computations through varied input connectivity; and evolutionary computation from computer science, where architecture is discovered through selection rather than hand-designed [30, 87].

Their synthesis produced the core GEENNS insight: *intelligence is not a single system but a collaboration of specialists*, and the collaboration pattern must itself be evolvable. GEENNS proposes to evolve the specialists (grids), freeze them, and then evolve the collaboration (mappers) for each new task.

7.1.1 The Seven Hypotheses

GEENNS rests on seven hypotheses. Two of them (H5 and H7) form the conceptual core; the remaining five concern substrate-level prerequisites tested in this thesis (H1–H4) and a multi-task mechanism validated in companion manuscripts (H6).

H1. The hyperparameter space of adaptive substrate neuroevolution can be navigated efficiently without sacrificing solution quality (TRQ1; Chapter 3).

H2. Adaptive substrates extend to high-dimensional inputs through spatial decomposition that overcomes central-bias representational collapse (TRQ2 and TRQ4; Chapter 4).

H3. ES-HyperNEAT's quadtree-based substrate discovery is structurally incompatible with GPU parallelism and must be replaced rather than optimized (TRQ2; Chapter 5).

H4. Adaptive substrate discovery can be reformulated for massively parallel execution without sacrificing adaptivity (TRQ3; Chapter 6).

H5. Consensus of distributed specialists (multiple grids, each processing partial information, combined through evolved coordination) produces more generalizable solutions than monolithic approaches.

H6. Neuromodulatory gain modulation, encoded as per-task signals that scale the influence of synaptic connections, enables a single substrate topology to support multiple tasks without catastrophic forgetting.

H7. Specialization of intelligence, where different computational units develop expertise in different domains, emerges naturally under evolutionary pressure with compositional task structure.

H1–H4 are answered in this thesis. H5 and H7 are partially addressed: the prototype (Section 7.6) demonstrates specialist composition at 5-grid scale but not the emergent specialization or robust zero-shot generalization that the hypotheses predict. H6 is answered in companion manuscripts that build on the thesis infrastructure (see Chapter 8 Table 8.5).

7.1.2 Research Scope

The research program comprises nine tasks organized into three phases. Table 7.1 maps each task to its status in this thesis, illustrating both the narrowing of scope and the deepening of substrate-level investigation.

7.1.3 From Breadth to Depth

The research program initially envisioned a broader cognitive-scale system. The thesis delivers something narrower but more rigorous: a substrate-level engineering investigation that removes the computational barriers blocking adaptive substrate neuroevolution. The nine tasks collapsed into a prerequisite stack: before we could pursue substrate expressiveness (Task 4), multi-task operation (Task 5), or lifelong learning (Task 9), we had to solve the computation problem (Task 7) and validate the spatial decomposition principle (Task 6). EMR-HyperNEAT's speedup has since opened Tasks 4 and 5 for companion manuscripts (per-node activation and aggregation address substrate expressiveness, and neuromodulation addresses multi-task operation), while Task 9 (lifelong learning) and the GEENNS-level integration (Task 8) remain open. The prototype reported in this chapter provides initial evidence that the compositional architecture is viable.

7.2 From Substrate Engineering to Composition

Three discoveries shaped the thesis trajectory: the central bias (Chapter 4), the quadtree–GPU incompatibility (Chapter 5), and the EMR reformulation that resolved it (Chapter 6). The combined effect reduces the GEENNS pipeline from computationally infeasible to tractable (Table B.3 in Appendix B).

The prototype (Section 7.6) validates the compositional architecture up to 5-grid Boolean scale (100% composition vs. 0% for every monolithic baseline) and up to 3-gate continuous composition, with the scaling limit at 4-gate multiplicative composition localized to the operator rather than the substrate; the thesis provides the engineering foundations required to pursue it at larger scale. The full research trajectory is presented in Appendix A.

Table 7.1: Research tasks and their thesis disposition. "Thesis" indicates a contribution presented in a thesis chapter; "Companion" indicates a direction validated in a companion manuscript that builds on the thesis infrastructure but is not part of this thesis (see Chapter 8 Table 8.5); "Future" indicates a direction the thesis infrastructure opens but that no companion manuscript yet addresses.

#	Original Task	Thesis Status	Location
1	NEAT/HyperNEAT foundation	Completed	Ch 2
2	ES-HyperNEAT	Completed + diagnosed	Ch 5
3	Hyperparameter optimization	Completed (TPE, 29% MNIST)	Ch 3
4	Substrate expressiveness	Companion (per-node activation/aggregation)	§8.4
5	Multi-task substrates	Companion (neuromodulation)	§8.4
6	Spatial decomposition	Completed (partitioned experts, 43% MNIST)	Ch 4
7	Tensor reformulation for substrate discovery	Completed (EMR-HyperNEAT)	Ch 6
8	Compositional architecture	Proof-of-concept validated	Section 7.6
9	Lifelong learning	Future	§8.4

7.3 Biological Inspiration

GEENNS draws inspiration from three principles: neocortical columnar organization, evolutionary architecture search, and emergent modularity under evolutionary pressure. We extract the computational insight from each and distinguish what is borrowed from biology from what is a computational design decision. GEENNS is inspired by the neocortex; it does not simulate the neocortex. A fourth biological principle, prior to these three, motivates the encoding itself and is treated in §1.1: DNA compresses the phenotype's construction rule into a genome orders of magnitude smaller than the phenotype it expresses, and indirect encoding in neuroevolution is the computational analog of that asymmetry. The present section concerns organizational principles (how the substrate is arranged and reused), not the encoding principle (how it is generated); the two operate at different levels and compound rather than substitute.

7.3.1 Columnar Organization: The Extracted Principle

The mammalian neocortex is organized into columns of approximately 0.5 mm diameter, each sharing the same canonical microcircuit [69]. This pattern arises independently across mammalian lineages [48], suggesting a fundamental organizational advantage: as Hawkins [35] argued, a repeated computational template, wired differently, produces functional diversity without a unique design per region. (Additional biological context appears in Appendix A.)

GEENNS does not imitate the neocortex. Mammalian columns have approximately six layers because biological evolution optimized for axon propagation speed, metabolic cost, physical volume, and ion-channel dynamics [64]. Machine substrates face different constraints (SIMT parallelism in §2.4, memory bandwidth, cache locality), so GEENNS prescribes no fixed layer count. The canonical microcircuit is a NEAT genome: evolution finds the optimal column template for the target hardware.

The extracted principle is *architectural homogeneity with functional diversity through connectivity* (Figure 7.1). All columns share the same evolved internal structure; different input routing patterns produce different functional behaviors. HyperNEAT applies this at the connection level (a single Compositional Pattern-Producing Network (CPPN) generates all weights); GEENNS applies it at the structural level (a single evolved template instantiated across all columns).

The case for evolutionary architecture search over hand-design, including the universal approximation guarantee and the combinatorial argument for evolved substrates, is presented in Appendix B.

7.3.2 Emergent Modularity Under Evolutionary Pressure

Kashtan and Alon [49] showed that modularly varying environments, where sub-problems are reused across changing goals, spontaneously produce modular network structure. Clune et al. [19] extended this: a connection-cost penalty produces modularity even in fixed environments. The implication is that compositional task structure should produce specialized, reusable grids without an explicit modularity objective.

GEENNS prescribes *structural* modularity (separate grids with separate CPPNs); whether grids develop *functional* modularity supporting compositional reuse depends on evolutionary dynamics. Prototype evidence (Section 7.6) is consistent: specialist grids

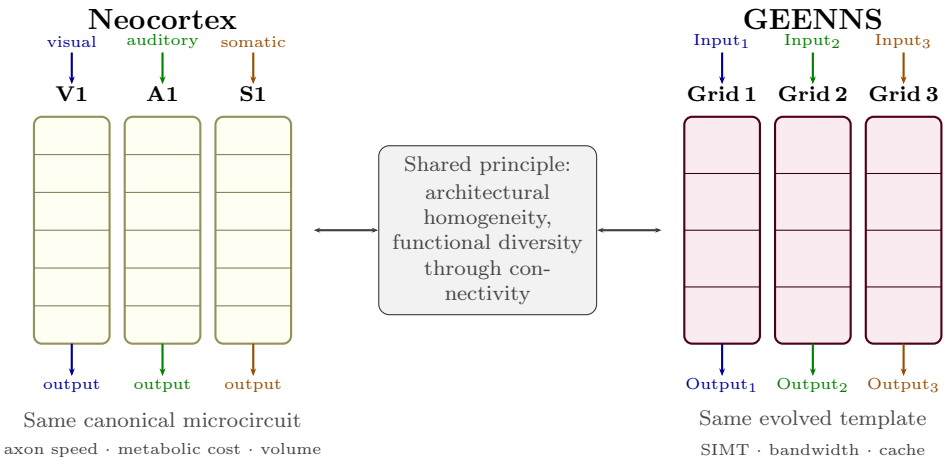

Figure 7.1: The columnar principle shared between neocortical and GEENNS organization. Left: three cortical areas (visual, auditory, somatosensory) share the same six-layer canonical microcircuit but produce different functions through different afferent connectivity. Right: GEENNS columns share the same NEAT-evolved template but produce different specialist behaviors through different evolved input routing patterns.

compose to solve 3-gate compound tasks with 100% success versus $\leq$3.3% for uniform-activation monolithic baselines, and the gap becomes decisive at 5-grid scale where every monolithic baseline (including the dynamic per-node function baseline) collapses to 0%. The concrete experimental signature is transfer without retraining (Figure 7.2): frozen modules compose into new combinations without weight modification. The prototype validates this up to 5-grid Boolean scale; whether it extends to harder, non-conjunctive domains is an empirical question for future work.

7.3.3 From Mimicry to Emergence

Biological inspiration provides constraints (the columnar principle, the template-reuse motif), but specific instantiations should emerge from evolution, not analogy. The shift this implies is from prescription to emergence.

The evidence is distributed across thesis contributions. We do not prescribe six layers; we evolve the layer count (Section 7.4.2). We do not prescribe how grids communicate; we evolve mappers that discover routing patterns (Section 7.4.3). EMR-HyperNEAT is a computational design, not biological mimicry; per-node function evolution, realized as additional CPPN output dimensions, is explored in companion manuscripts that build on the thesis infrastructure (see Chapter 8 Table 8.5), with no biological constraint on which functions are selected.

The prototype (Section 7.6) illustrates this concretely. The compound task $\text{XOR}(a, b) \wedge \text{AND}(c, d) \wedge \text{OR}(e, f)$ is solved because evolution discovered that routing through frozen specialists produces correct outputs, not because composition was designed in. This is the central insight: shift from "copy the brain's structure" to "create the brain's conditions for emergence." The limitation is equally important: the emergence observed is task-specific, not general. Demonstrating that these conditions reliably produce emergent compositional reasoning remains the open problem.

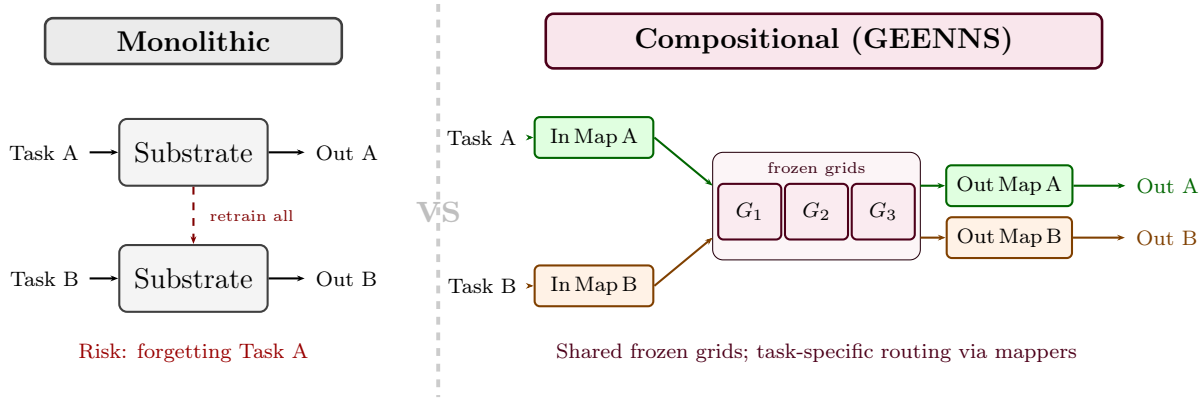

Figure 7.2: Monolithic versus compositional substrate design. Left: a monolithic substrate handles each task, requiring full retraining and risking catastrophic forgetting when tasks change. Right: GEENNS organizes frozen specialist grids and evolves task-specific input and output mappers. New tasks require only new mapper pairs; grids retain their specialized capabilities across all tasks.

7.4 GEENNS Algorithm Design

The following sections present the GEENNS algorithm as a *design specification*: a target architecture motivated and enabled by the substrate infrastructure developed in Chapters 3–6, not a description of implemented behavior. No implementation or experimental validation exists for this design beyond the prototype evidence in Section 7.6; all plasticity mechanisms, developmental stages, and the multi-level CPPN hierarchy are theoretical proposals that serve as a roadmap for future implementation.

GEENNS aims to enable compositional reasoning through the architectural separation of computational components (grids) and composition logic (mappers). The rest of this section distinguishes what is designed (below), what is implemented (Chapters 3–6), and what remains theoretical (the three-type mapper formalism with explicit inter-grid routing, plasticity).

7.4.1 Core Architectural Principles

GEENNS rests on five design principles, each motivated by a specific observation about biological or computational intelligence:

1. **General-purpose over task-specific.** GEENNS evolves substrates, not solutions. A grid is not trained to classify digits or balance poles; it is evolved to perform a class of computations (feature extraction, transformation, integration) that can be composed for many tasks. This principle addresses the hypothesis that intelligence is a set of specialized capabilities, not a single general capability (H7).

2. **Indirect over direct encoding.** Connectivity patterns are generated by CPPNs from spatial coordinates, exploiting geometric regularities (symmetry, repetition, locality). A compact genome thus encodes a large-scale structured substrate. This principle is inherited from HyperNEAT, refined in ES-HyperNEAT, and extended in EMR-HyperNEAT (Chapter 6).

3. **Developmental over static.** The substrate is not a fixed graph. It forms through a staged developmental process (node generation, layer organization, connection formation, refinement, and operational maturation), such that the same genome reliably produces the same phenotype. This principle is inspired by biological neurodevelopment, where a compact genome orchestrates the construction of a brain containing billions of connections.

4. **Adaptive over fixed.** In the full GEENNS design, connections would be subject to plasticity: Hebbian strengthening, homeostatic regulation, structural pruning, and neuromodulatory gating. This would enable the substrate to adapt during operation without requiring re-evolution. Three of these (Hebbian, homeostatic, structural) remain future work; neuromodulatory gating has been demonstrated in companion work (see Chapter 8 Table 8.5).

5. **Modular over monolithic.** Components function independently or together. A single grid can solve a task. Multiple grids can be composed through mappers to solve tasks that require capabilities beyond any single grid. This principle implements the distributed-specialist hypothesis (H5).

7.4.2 The Columnar Substrate

A grid is a two-dimensional substrate containing columns and layers. Each column is a vertical processing unit; each layer is a horizontal functional division, with number and function evolved rather than predetermined. Nodes are addressed by (c, l, n): column index, layer index, and node index within the layer.

The key structural feature is the *canonical microcircuit template*: all columns within a grid share the same internal structure, defining layer organization, node properties, and intra-columnar connectivity. The template is evolved via NEAT [87], applying the columnar principle from Section 7.3.1: the same circuit, replicated across a grid, produces different functions depending on input connectivity.

What "frozen" means. A *frozen* grid has its *columnar scaffold* fixed (layer count, inter-layer connectivity, inter-column topology, and node placement) after the developmental phase. This parallels neocortical development, where the large-scale structural scaffold stabilizes after neurogenesis and pruning while synaptic plasticity continues [69]. In the full design, frozen grids retain potential for weight-level plasticity (Hebbian, homeostatic, or neuromodulatory; Appendix B), and new columns can be instantiated from the same template without modifying existing ones. In the current prototype (Section 7.6), "frozen" means fully static; scaffold-frozen, plasticity-active grids are future work.

Grids are evolved using EMR-HyperNEAT (Chapter 6), which generates the connectivity patterns from CPPN queries over normalized spatial coordinates. Each node could have an evolved activation function selected from a palette of candidate functions and an evolved aggregation function that determines how incoming signals combine (§8.4). Task-specific behavior within a shared grid topology could be achieved through neuromodulatory gain modulation.

Whether evolution actually discovers grids that specialize in interpretable ways remains an open question. The partitioned-experts experiments in Chapter 4 provide partial evidence: spatially partitioned experts do develop distinct specializations (peripheral vs.

central pixel processing), though at a much simpler level than the full grid specialization envisioned here.

The full architecture (minicolumns, columns, and multi-level CPPNs) is detailed in Appendix B.

7.4.3 Mappers as Compositional Coordinators

The mapper approach is the core mechanism for compositional reasoning in GEENNS, with zero implementation beyond the prototype. We elaborate the design here because every substrate-level barrier we solve is a prerequisite for mappers to function.

Mappers are evolved functions that coordinate how grids are used for a given task, in three types:

- **Input mappers** decompose a task's input into grid-appropriate queries; the mapper does not *know* what each grid does but evolves to discover which routing patterns produce correct outputs.
- **Output mappers** compose grid responses into task solutions by extracting and combining relevant signals.
- **Inter-grid mappers** route information between grids during processing, enabling compositional depth (e.g., a transformation grid's output feeding an integration grid).

Each mapper is evolved via NEAT. For a new task, only mappers are evolved; grids remain frozen. Although the input and output mappers are described separately above, their fitness signals are necessarily coupled: routing is only judged successful when the composition recovers the right answer, so the two cannot be evolved independently. The prototype realizes this coupling within a single NEAT mapper that receives both raw inputs and grid outputs, treating input-routing and output-composition as functional regions of one evolved network; the full GEENNS design retains the option of separating them into distinct co-evolving populations. This is the proposed lifelong learning mechanism: task-specific routing without substrate modification.

The critical question, whether this works, is partially answered by the prototype (Section 7.6). The risks are substantial: mapper complexity might grow exponentially with task count, evolution might not discover generalizable decomposition strategies, and routing overhead might be prohibitive.

End-to-end walkthrough. Consider $\mathrm{XOR}(a, b) \wedge \mathrm{AND}(c, d) \wedge \mathrm{OR}(e, f)$: three Boolean operations on independent input pairs whose results are combined. The pipeline:

1. **Specialist training.** Three grids (G_{XOR}, G_{AND}, G_{OR}) are independently evolved on their respective gates, then frozen.
2. **Input mapper.** A NEAT-evolved mapper routes (a, b) to G_{XOR}, (c, d) to G_{AND}, and (e, f) to G_{OR}.
3. **Grid processing.** Each frozen grid processes its pair independently.
4. **Output mapper.** A NEAT-evolved mapper composes the three grid outputs into the final conjunction.

This achieves 100% success at 3-grid scale in the prototype. Figure 7.3 illustrates the full pipeline.

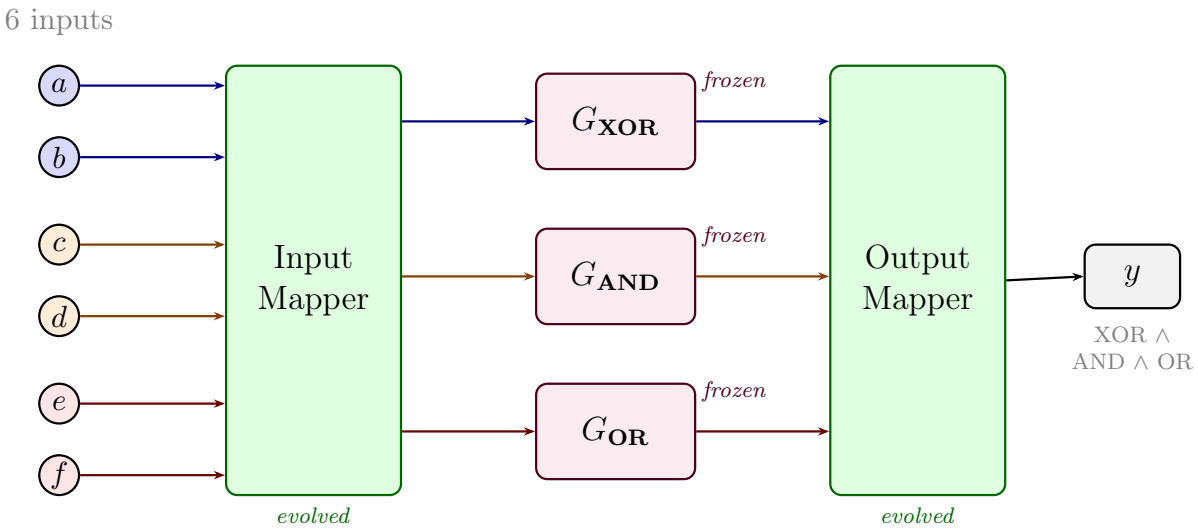

Figure 7.3: End-to-end compositional routing for $\text{XOR}(a, b) \wedge \text{AND}(c, d) \wedge \text{OR}(e, f)$. Six Boolean inputs enter the evolved input mapper (green), which routes each pair to the appropriate frozen specialist grid (purple). Each grid processes its inputs independently. The evolved output mapper (green) composes the three grid outputs into the final compound result. Only the mappers are evolved per task; grids are trained once and reused.

The full GEENNS specification includes additional detail: a multi-level CPPN hierarchy for connectivity at different spatial scales (Table B.2), a five-stage developmental pipeline (Figure B.6), four plasticity mechanisms, a comparison with established continual learning methods (Table B.4), and a formal mathematical specification (Definitions B.1–B.2). These are presented in Appendix B as a design specification; none have been implemented or validated beyond the prototype. The remainder of this chapter focuses on the substrate barriers that must be solved before these mechanisms can be pursued, followed by the prototype evidence.

A comparison of six substrate evolution approaches (Table B.1, Appendix B) confirms that EMR-HyperNEAT is the only method providing both spatial organization and per-node feature evolution at practical cost. The analysis of CoDeepNEAT, the closest conceptual prior art, is presented alongside the formal specification.

7.5 Why the Substrate Problem Must Be Solved First

The GEENNS architecture described in the previous section requires substrates that are (1) computationally tractable to evolve, (2) expressive enough at the function level to support diverse computations, and (3) capable of multi-task operation. ES-HyperNEAT (the canonical adaptive substrate method) fails on all three counts. This section characterizes each failure mode and maps it to the thesis Part that addresses it. "Expressiveness" here is a GEENNS-prerequisite label distinct from the central-bias-driven "Representational Collapse" of Chapter 4: they concern function diversity and spatial coverage respectively, and are independent barriers.

7.5.1 Barrier 1: Computational. ES-HyperNEAT Cannot Scale

ES-HyperNEAT discovers substrate topology through a quadtree that recursively subdivides the spatial domain, retaining regions where the CPPN output shows high variance (indicating information content) and pruning regions where the output is uniform (indicating redundancy). This adaptive discovery mechanism is the algorithm's defining feature: it determines not just connection *weights* but connection *placement*, enabling the substrate to match the problem's structural complexity.

The quadtree is also the algorithm's defining limitation. Each subdivision decision depends on the variance of the CPPN output evaluated at four probe points inside the current cell (the centers of its four child sub-quadrants) whose coordinates are determined by the previous subdivision. This sequential dependency chain prevents parallelization. While direct-encoding methods like NEAT can evaluate entire populations in parallel on a GPU (because each individual's fitness evaluation is independent), ES-HyperNEAT must construct each individual's substrate sequentially because the quadtree decisions are individual-specific.

The quantitative impact: our benchmarks show that ES-HyperNEAT's substrate construction time grows exponentially with resolution depth, while GPU-parallelized evaluation would remain approximately constant up to the hardware's memory limit. At depth 7 (21,845 candidate positions), the sequential construction dominates total runtime. A one-way ANOVA with depth as factor yields $F(6, 449) = 87.04$, $p < 10^{-71}$, $\eta^2 = 0.54$ (with $R^2 = 0.95$ from a separate regression fit), confirming that depth (and therefore sequential construction cost) is the primary determinant of runtime.

For GEENNS, this barrier is prohibitive. At a modest (k=5, T=10) scale the architecture requires 34 CPPNs (Appendix B), each needing thousands of optimization trials, and cannot use an algorithm that runs each trial on CPU. The computation time is measured in CPU-days to years, not hours: roughly 250 CPU-days at XOR scale, rising to years for realistic higher-dimensional tasks. This estimate extrapolates from XOR-scale benchmarks (2-input, depth 4–7) to the 34-CPPN pipeline with substantially higher dimensionality. The extrapolation is illustrative rather than precise. Actual GEENNS-scale costs would depend on per-CPPN complexity, parallelism across CPPNs, and hardware configuration, but the order-of-magnitude gap between years (ES-HyperNEAT) and days (EMR with pruner) is robust to reasonable variation in these factors.

Thesis response. Chapters 5 and 6 remove this barrier. Chapter 5 proves the incompatibility of ES-HyperNEAT's quadtree with GPU acceleration. Chapter 6 introduces EMR-HyperNEAT, which achieves up to 34× per-generation GPU speedup on XOR at depths 5–7 (population 1000) by replacing the sequential quadtree with a parallel evaluate-then-filter pipeline. Chapter 3 addresses the hyperparameter explosion via TPE optimization and an early-stopping pruner that reduces computational cost by 41.6%.

7.5.2 Barrier 2: Expressiveness. Substrates Lack Function Diversity

Standard indirect-encoding neuroevolution assigns a *single* activation function to all nodes in the substrate. This is the default behavior in HyperNEAT, ES-HyperNEAT, and their implementations in frameworks like PUREPLES. The choice of activation function is typically made by the experimenter before evolution begins, based on problem-domain intuition.

This uniform-function assumption is limiting. A formal argument shows that substrates using only monotonic activation functions (sigmoid, tanh, ReLU) cannot converge on parity problems under indirect encoding, regardless of topology or training duration. The result is mathematical, not empirical: indirect encoding's symmetric weight-generation mechanism, combined with a monotonic activation function, produces substrates whose decision boundaries cannot partition the input space into the checkerboard pattern required by parity.

The practical consequence is that the choice of a single activation function can make the difference between convergence in 1 generation (with `sin`) and zero convergence (with `tanh`). Per-node activation function evolution, demonstrated in a companion manuscript, lifts this constraint at negligible marginal cost via an additional CPPN output dimension.

For GEENNS, this barrier means that grid specialists cannot compute everything they need to compute. A grid that uses only `tanh` cannot solve parity-like sub-problems. A grid that uses only `sin` may struggle with threshold-based classification. Compositional intelligence requires grids with diverse computational capabilities, which in turn requires per-node function diversity.

Thesis enabler. EMR-HyperNEAT's batch CPPN evaluation makes per-node function assignment computationally feasible at negligible additional cost (Chapter 6). Per-node activation, per-node aggregation, and bio-inspired palette meta-learning (strategies for evolving which activation and aggregation functions are available to nodes across generations) are demonstrated in companion manuscripts that build on this infrastructure; the thesis itself neither implements nor validates these results but is the prerequisite that made them feasible.

7.5.3 Barrier 3: Multi-Task. Substrates Cannot Handle Multiple Tasks

A single substrate evolved for one task cannot solve a different task without re-evolution. This is not a weakness of neuroevolution specifically; it is the fundamental problem of catastrophic forgetting [31, 65] expressed in evolutionary terms. If we evolve a substrate for Task A and then evolve it further for Task B, the weights that made Task A work are overwritten by the weights that make Task B work.

Standard continual learning methods from the deep learning literature (Elastic Weight Consolidation (EWC) [51], Progressive Neural Networks [80], PackNet [62]) assume gradient-based training on fixed-topology networks. None of them directly applies to evolved substrates, where the topology itself is the product of evolution and where there are no gradients to regularize.

For GEENNS, this barrier is structural. The entire multi-grid architecture assumes that grids can serve multiple tasks simultaneously: a feature extraction grid that processes visual input for both classification and detection, a transformation grid that performs rotations for both image alignment and spatial reasoning. If each task requires a completely separate substrate, then multi-grid composition offers no advantage over simply evolving separate substrates per task.

Thesis response. Chapter 4 demonstrates spatial decomposition through partitioned experts, achieving 43% MNIST accuracy (105% improvement over baseline) and providing preliminary evidence for the spatial decomposition principle underlying GEENNS. Two companion manuscripts address this barrier beyond the thesis: one demonstrates gain modulation for multi-task operation from a single topology; the other characterizes the

population-alignment problem that arises when neuromodulatory signals are co-evolved alongside substrate topology.

7.5.4 The Three Barriers as Thesis Structure

Table 7.2 summarizes the mapping from barriers to thesis contributions. The structure reflects the chronological order in which the solutions were developed.

Table 7.2: Mapping from GEENNS barriers to thesis contributions. Each barrier prevents a specific GEENNS capability; each contribution provides part of the solution. "Thesis" indicates a contribution presented in a thesis chapter; "Companion" indicates a direction validated in a companion manuscript that builds on the thesis infrastructure but is not part of this thesis; see Chapter 8 Table 8.5 for the full companion-work status summary.

Barrier	Problem	Contribution	Key Result
Computational	ES-HyperNEAT quadtree is sequential; GPU parallelism impossible	Thesis: Ch 5 (GPU incompatibility)	GPU incompatibility proof
		Thesis: Ch 6 (EMR-HyperNEAT)	12–34× per-gen GPU speedup (XOR, D5–7, pop 1000)
		Thesis: Ch 3 (TPE + pruner)	41.6% cost reduction
Expressiveness	Uniform activation and aggregation limit substrate expressiveness	Companion: per-node activation	Per-node activation via CPPN output
		Companion: per-node aggregation	Per-node aggregation via CPPN output
		Companion: bio-inspired palettes	Meta-learning of function palettes
Multi-task	No mechanism for task-specific behavior in shared substrates	Thesis: Ch 4 (partitioned experts)	43% MNIST (105% ↑)
		Companion: neuromodulation	Gain modulation for multi-task
		Companion: co-evolving NT signals	Population-alignment problem characterized

The three barriers are not independent. Solving the computational barrier enables the experiments that reveal the expressiveness barrier: per-node function evolution would require thousands of evolutionary runs that would be impractical on ES-HyperNEAT. Solving the expressiveness barrier would enable multi-task operation: neuromodulation would require per-task activation function selection, which requires per-node function diversity. And the multi-task solutions validate the grid specialization concept central to GEENNS's compositional architecture.

The thesis thus follows a linear dependency chain: computation → expressiveness → multi-task → composition. Each link is necessary. Removing any one would leave the subsequent links without a foundation.

7.6 Prototype Evidence

We validated a proof-of-concept prototype across more than 40 experimental conditions and approximately 3,100 runs. The 5-grid Boolean scaling experiment alone accounts for

∼806 CPU-hours owing to its 1024-row truth table and 2000-generation budget; the 6-grid Boolean and continuous additive-scaling extensions add comparable wall-clock cost, while the remaining conditions together account for a small fraction of the total. The prototype tests the core GEENNS pipeline (specialist grid training, freezing, and mapper-based composition) on compound Boolean tasks, with a continuous-gate extension and a multi-task amortization study. We report results, confounds, and limitations. This section presents the evidence relevant to the thesis narrative.

7.6.1 Experimental Design

The prototype is enabled by the EMR-HyperNEAT tensor reformulation of Chapter 6: without uniform-shape substrate tensors, evolving one specialist grid per gate plus a mapper population on top would be computationally out of reach. Given that infrastructure, the pipeline proceeds in two stages. First, an EMR-HyperNEAT substrate is evolved for each individual Boolean task (XOR, AND, OR) and frozen on convergence. Second, a NEAT-evolved mapper population is trained to route raw task inputs through the frozen grids and combine the resulting grid outputs into a compound prediction (e.g., $\mathrm{XOR}(a, b) \wedge \mathrm{AND}(c, d) \wedge \mathrm{OR}(e, f)$). Every mapper individual receives the raw inputs and all grid outputs; no architectural constraint forces it to use the grids.

Substrate configuration. All conditions share the EMR-HyperNEAT substrate parameters in Table 7.3. These settings follow directly from the EMR-HyperNEAT system validated in Chapter 6: depth 4 provides sufficient resolution for Boolean tasks without incurring the exponential node growth that higher depths introduce, and the variance/division thresholds (0.03) match the values used throughout the cross-domain validation in Section 6.5.9. NEAT-specific parameters (species count, compatibility threshold, mutation rates, crossover rate, elitism, CPPN architecture) use TensorNEAT defaults throughout.

Table 7.3: Shared EMR-HyperNEAT substrate parameters for all prototype conditions.

Parameter	Value
Initial depth	0
Max depth	4
Variance threshold	0.03
Division threshold	0.03
Band threshold	0.3
Max weight	3.0
Success threshold	$f \geq 0.95$
Output activation	sigmoid
Output node	single at (0.0, 1.0)

Substrate coordinate layout. Raw inputs are placed at $y = -1.0$, evenly spaced along the x-axis; bias is co-located at $(0, -1.0)$. In augmented (compositional) conditions, frozen grid outputs are placed at $y = -0.5$, an intermediate coordinate that gives the CPPN spatial information to differentiate raw input signals from pre-solved grid outputs. The single output node sits at $(0, 1.0)$. Figure 7.4 contrasts the monolithic and compositional layouts for a 2-grid task.

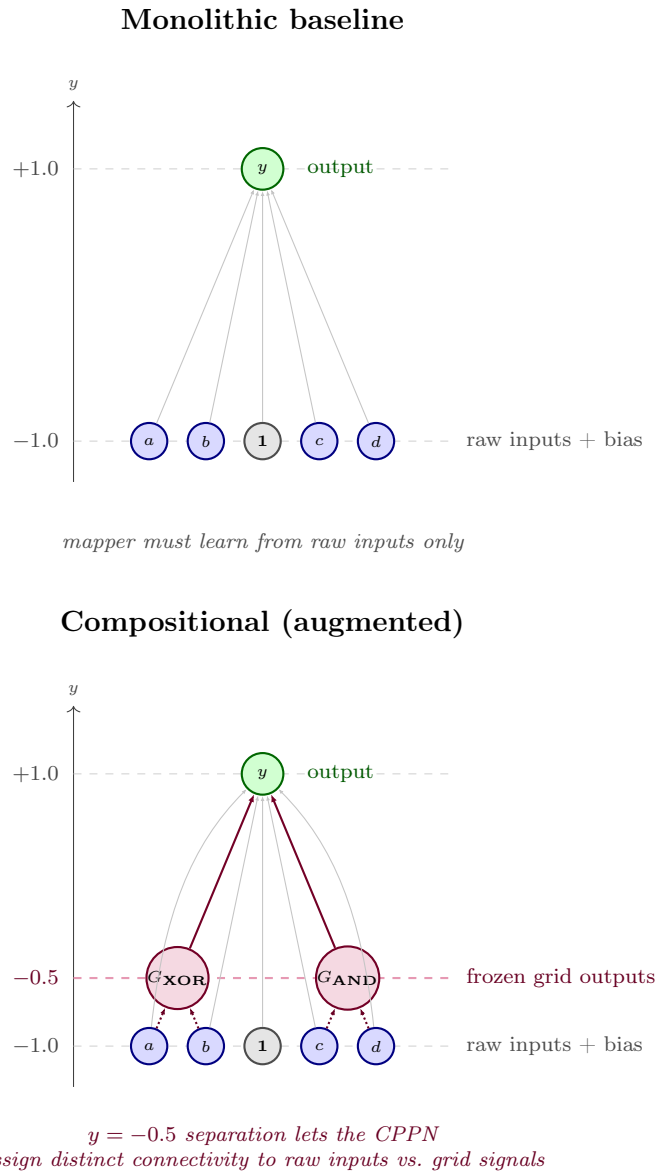

Figure 7.4: Substrate coordinate layout for monolithic (top) and compositional (bottom) conditions, illustrated for a 2-grid task (four raw inputs a, b, c, d plus bias). Raw inputs sit at $y = -1.0$ evenly spaced along the x-axis, with bias co-located at $(0, -1.0)$. The single output node sits at $(0, +1.0)$. In compositional conditions, frozen grid outputs (G_{XOR}, G_{AND}) occupy an intermediate $y = -0.5$ plane; this coordinate separation gives the CPPN a spatial cue for treating raw inputs and pre-solved grid signals differently, rather than requiring the mapper to disentangle them from a single shared layer. Three arrow styles appear in the bottom panel: thin gray arrows denote the mapper's evolved raw-input and bias connections to the output (curved for a and d to route around the grid-output nodes); thick purple arrows denote the mapper's evolved grid-output connections; densely dotted arrows denote the frozen grids' fixed input dependencies (a, b feed G_{XOR}; c, d feed G_{AND})—pre-computations external to the mapper substrate, not evolved.

Frozen grid forward pass. Once a specialist grid is trained and frozen, its forward pass reduces to a fixed two-layer computation: output $= \sigma(\phi(\mathbf{x}\mathbf{W}_1)\mathbf{W}_2)$, where $\mathbf{W}_1$ and $\mathbf{W}_2$ are the weight matrices generated by the CPPN during evolution, ϕ is the grid's hidden activation (sin for XOR, tanh for all others), and σ is sigmoid. At inference, grid evaluation is a pair of matrix multiplications with no evolutionary overhead. Fitness is $f = 1 - \text{MSE}$ over all input patterns; a run succeeds when $f \geq 0.95$. Success thresholds tighten at larger grid counts to maintain margin above the class-imbalance floor: $f \geq 0.99$ at 5-grid (where the all-zeros predictor already scores $f = 0.982$) and $f \geq 0.995$ at 6-grid (floor $f = 0.9912$); the per-scale justifications appear in §7.6.3 and §7.6.2.

Pipeline architectures and activation modes. Two pipeline architectures and three hidden-activation modes are compared.

Compositional (augmented) pipelines. A mapper substrate combines raw inputs with outputs from frozen specialist grids (each grid has its own pre-trained activation: sin for XOR, tanh for others). The mapper's hidden layer is fixed at tanh via the *disabled* mode (per-node activation-selection turned off); per-task activation diversity is handled externally by the frozen grids.

Monolithic baselines. A single substrate is used, with no frozen grids. The hidden layer is configured in one of two ways:

- *uniform* (`global`): a single activation (sin or tanh) is assigned uniformly to every hidden node: the mono-sin and mono-tanh baselines.
- *dynamic* (`cppn_output`): an extra CPPN output selects from a 6-function palette per node: the `-dyn` baseline. The dynamic function mechanism is the subject of work beyond this thesis; it is included here solely as a diagnostic baseline to isolate whether per-node function selection can substitute for compositional architecture (PQ7). Dynamic function evolution is not a contribution of this thesis.

The *disabled* mode (compositional mapper, tanh everywhere) and the *global*+tanh mode (monolithic baseline, tanh everywhere) are observationally identical at the hidden-layer level. They are kept as separate labels because they apply to architecturally different pipelines: the disabled-mode mapper is one component of a compositional pipeline that also contains frozen specialist grids supplying per-task signals; the global+tanh monolithic baseline has no frozen grids.

Population and generation budgets. Table 7.4 summarizes the evolutionary budgets. Specialist grids use smaller populations (150) and shorter runs (200 generations) because single Boolean gates are simple tasks that converge reliably. Mapper and monolithic conditions use population 300 and 500 generations. The 4-grid conditions double the generation budget to 1,000 to accommodate the 256-row truth table; the 5- and 6-grid conditions raise it again to 2,000 for their 1024- and 4096-row truth tables.

Problem scaling. As grids are added, the mapper's input dimensionality and the truth table both grow (Table 7.5). Compositional conditions receive additional grid-output dimensions but benefit from the $y = -0.5$ coordinate separation that allows the CPPN to assign distinct connectivity patterns to raw inputs versus grid signals. Monolithic baselines receive only the raw inputs and must discover the correct spatial partitioning of activation functions internally.

Table 7.4: Population and generation budgets by experimental scale.

Scale	Pop	Max Gen	Notes
Grid specialists	150	200	XOR uses sin; AND, OR, NAND, NOR use tanh
1-grid conditions	300	500	
2-grid conditions	300	500	Composition-only oracle uses pop 150
3-grid conditions	300	500	
4-grid conditions	300	1000	Doubled for 256-row truth table
5-grid conditions	300	2000	Quadrupled for 1024-row truth table
6-grid conditions	300	2000	4096-row truth table; threshold $f \geq 0.995$

Table 7.5: Input dimensionality and truth-table size by grid scale. "Comp" = compositional (raw + grid outputs + bias); "Mono" = monolithic (raw + bias).

Scale	Task	Raw	Truth table	Comp inputs	Mono inputs
1-grid	XOR	2 + bias	4 rows	4	3
2-grid	XOR∧AND	4 + bias	16 rows	7	5
3-grid	XOR∧AND∧OR	6 + bias	64 rows	10	7
4-grid	XOR∧AND∧OR∧NAND	8 + bias	256 rows	13	9
5-grid	XOR∧AND∧OR∧NAND∧NOR	10 + bias	1024 rows	16	11
6-grid	XOR∧AND∧OR∧NAND∧NOR∧XNOR	12 + bias	4096 rows	19	13

The experiments address seven prototype questions, labeled PQ1–PQ7 to mark them as scope-local rather than thesis-wide research questions (TRQ1–TRQ4 in Chapter 1):

PQ1 (Feasibility) Do EMR-HyperNEAT grids converge reliably enough to serve as frozen specialist modules?

PQ2 (Composition) Can a mapper learn to compose pre-solved grid outputs into a compound solution?

PQ3 (Routing & efficiency) Does the compositional pipeline improve success rate and convergence speed over monolithic evolution, controlling for activation function?

PQ4 (Grid utilization) Does the mapper genuinely use grid outputs, or does it bypass them to solve monolithically?

PQ5 (Generalization) Does the evolved mapper generalize to unseen compositions or grid permutations?

PQ6 (Reuse) Can the same frozen grids support mapper evolution for different compound tasks, and can architectural bias improve selective routing?

PQ7 (Scaling) Does the compositional advantage persist, and do monolithic baselines degrade, as task complexity rises across grid counts, and at what scale (if any) does composition become structurally necessary rather than optional? *Hypothesis*: a complexity threshold exists beyond which a monolithic substrate with per-node dynamic function

selection can no longer reliably discover the correct spatial partitioning of activation functions, and compositional architecture becomes structurally necessary. The prototype tests this across 1, 2, 3, 4, and 5 Boolean grids; 6-grid Boolean and continuous-gate extensions appear in §7.6.2 and §7.6.5.

Table 7.6 summarizes the key experiments, organized by the prototype question each addresses. Several of its labels refer to specific baseline configurations: a *composition-only oracle* feeds the mapper only frozen-grid outputs (no raw inputs); *noise control* experiments add task-irrelevant distractor input dimensions—extra inputs that probe whether the mapper genuinely uses grid outputs—either *constant* (the distractor is held fixed) or *resampled* (the distractor is drawn anew per evaluation); *gated* architectures impose a structural bias toward grid outputs.

7.6.2 What Works

Figure 7.5 summarizes success rates across the conditions plotted through 4-grid scale.

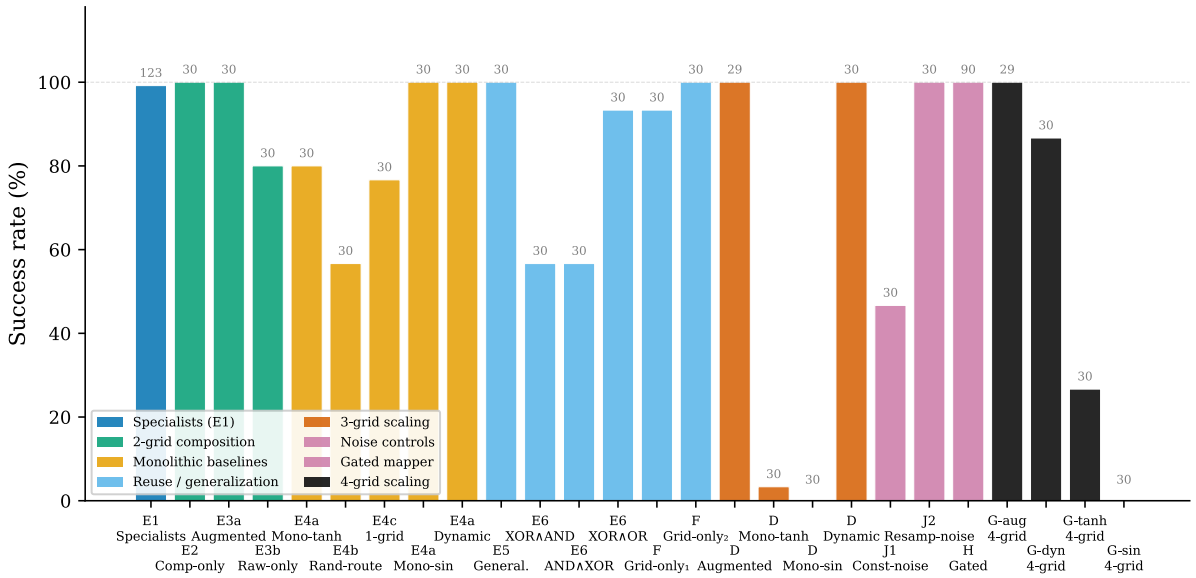

Figure 7.5: Success rates across the prototype's conditions through 4-grid scale ($N = 30$ seeds unless noted; 5-grid Boolean in Table 7.7; 6-grid Boolean and continuous gates in §7.6.2 and §7.6.5). Compositional pipelines achieve 100% at every scale shown. Dynamic function baselines match at 2- and 3-grid but degrade to 87% at 4-grid. Uniform-activation monolithic baselines collapse at 3+ grids. Gated selective routing lifts 2-grid frozen grid reuse rates from 57–93% to 100%. Multi-task amortization (Exp I; see Table 7.6 and §7.6.2) is not plotted here: its finding concerns amortization economics rather than per-scale success rates.

Specialist grids converge and compose. Specialist grid feasibility confirms reliable convergence (XOR, AND, OR, NAND, and NOR). At 3-grid scale, three-grid composition solving $\text{XOR}(a, b) \wedge \text{AND}(c, d) \wedge \text{OR}(e, f)$ achieves 100% success (see Table 7.7). Frozen grids support mapper re-evolution: the same three grids, without retraining, support three different compound tasks in the frozen grid reuse experiment with 57–93% success rates.

Table 7.6: GEENNS prototype experiments organized by prototype question (PQ). The "Code" column gives the short identifier used in the figure legends of Figures 7.5 and 7.6. The experiments listed here account for about half of the prototype's ∼3,100 runs; the remaining runs (5-grid and 6-grid Boolean scaling, continuous-gate extension through 8 gates, gated selective routing across 3–6 grids, the operator landscape control, and the warm-start transfer study) are summarized in the body of §7.6.

Code	Experiment	PQ	N	Key Result
PQ1–PQ3: Feasibility, composition, and routing				
E1	Specialist grid feasibility	1	150	100% convergence, all 5 gates
E2	Composition-only oracle	2	60	100%, median 44 gen
E3	Two-grid routing and composition	3	60	100% vs. 80% (raw-only ablation)
E4	Monolithic baselines (5 cond.)	3	150	dyn 100% at 3 gen; tanh 80%
PQ4: Grid utilization				
E6	Frozen grid reuse	4, 6	120	57–93%; noise control 70%
J1	Constant noise control	4	30	47%; non-informative dims hurt search
J2	Resampled noise regularization	4	30	100%; implicit regularization
PQ5–PQ6: Generalization and reuse				
E5	Compositional generalization	5	30	Fitness 0.53–0.80 (below 0.95)
F	Grid-only composition	6	60	93–100% without raw inputs
H	Gated selective routing	6	90	100% (90/90); 8.3× selectivity
PQ7: Scaling (1–4 grids enumerated here; 5- and 6-grid Boolean plus continuous gates in §7.6 body)				
D	Three-grid scaling	7	119	100% compositional; ≤3.3% mono; 100% dynamic
G-aug	Four-grid composition	7	29	100%, median 40 gen, 95% CI [34, 70]
G-dyn	Four-grid dynamic baseline	7	30	87% (26/30), median 321 gen
G-tanh	Four-grid mono-tanh baseline	7	30	27% (8/30), median 437 gen
G-sin	Four-grid mono-sin baseline	7	30	0% (0/30)
1grid-comp	Single-grid composition	7	30	100%, median 1 gen
1grid-dyn	Single-grid dynamic	7	30	100%, median 1 gen
1grid-tanh	Single-grid mono-tanh	7	30	37% (11/30), median 95 gen
1grid-sin	Single-grid mono-sin	7	30	100%, median 1 gen
I	Multi-task amortization	7	595	94.9% compositional vs. 93.3% dynamic; pipeline 1.34× slower at 10 tasks; break-even ∼80
	Total (this table)		**>1,700**	

The single-grid composition control (frozen XOR grid + raw inputs) converges in median 1 generation (30/30), confirming that frozen grid outputs provide immediately useful features that the mapper exploits without requiring substantial evolutionary search. At 4-grid scale, four-grid composition extends the pipeline to $\text{XOR}(a, b) \wedge \text{AND}(c, d) \wedge \text{OR}(e, f) \wedge \text{NAND}(g, h)$, an 8-input, 256-row truth table, and maintains 100% success (see Table 7.7). Convergence is actually *faster* at 4-grid than at 3-grid: the extra frozen grid appears to tighten the routing signal rather than add search burden.

Routing cost and amortization. Two-grid routing provides a 9× generation-count speedup over the 2-grid mono-sin baseline (monolithic, uniform sin activation; median 9 generations vs. 82 generations). The pipeline cost overhead is modest (1.04× per task), breaking even at approximately 2 tasks because specialist grids are trained once and reused at the 2-grid scale.

At larger multi-task scales the amortization story is more complicated. The multi-task amortization experiment sweeps 10 pairwise AND compositions across 595 runs and reports augmented success at 94.9% versus dynamic-function monolithic at 93.3%: reuse works, but at 10 tasks the full pipeline cost (specialist training plus mapper evolution across all 10 targets) is 1.34× the total monolithic cost, with a projected break-even near 80 reused tasks. At 10-task scale the pipeline is therefore negative on raw efficiency. The amortization argument regains traction at 5-grid scale, where the dynamic baseline falls to 0% and composition retains 100%: the pipeline's value at 5-grid is capability, not efficiency, and it solves tasks the monolithic alternative cannot. This reframes the amortization claim: at small grid counts the monolithic baseline remains competitive and composition's overhead dominates; beyond the complexity threshold the overhead becomes the cost of capability rather than a tax on reuse.

Mappers genuinely use grid outputs. Weight analysis confirms grid usage: grid-output connections are 1.25× stronger than raw-input connections ($p < 0.001$). A post hoc ablation sharpens this picture: zeroing the two grid-output columns of 30 converged mappers drops fitness by 0.088 ± 0.012 (from ∼0.965 to ∼0.877), whereas zeroing the four raw-input columns drops it by 0.364 ± 0.175 (to ∼0.601). Ablation therefore attributes roughly 4× more of the fitness to raw inputs than to grid outputs, even though the mapper's weights favor grid outputs. The two measurements are not at odds once the quantities are separated: weight magnitude tracks which inputs the mapper prefers to connect to, while ablation tracks which inputs the computation cannot function without. The grids do not replace the raw inputs; they supply pre-solved features that narrow the search region enough to explain the 9× convergence speedup relative to raw-only baselines, while the raw inputs continue to carry the full 2^4-pattern signal over which the mapper actually computes.

Gating elicits selective routing and extends composition's reach. Gated selective routing addresses the indiscriminate routing observed in the frozen grid reuse experiment by introducing evolved per-column sigmoid gates on each grid-output channel. With this architectural bias, all three 2-grid reuse tasks reach 100% versus the unconstrained 57–93% observed without gating. For each compound task, we label the task-relevant grids as *correct* and the remaining grids as *distractor*. Gate-ratio analysis reveals three qualitatively distinct routing strategies rather than a single selective pattern: one task produces a strongly selective gate signature (correct-to-distractor 8.28×, the strongest

direct evidence in the prototype of evolved selective routing under explicit architectural support), one converges with *inverted* gates (0.63×, distractors recruited as complementary features rather than suppressed, median 19 gen), and one is neutral (1.05×, median 4 gen, both grid outputs carrying the same signal). Architectural bias therefore elicits selective routing *when the task configuration admits it*, but the mapper can also solve compound tasks without favoring correct over distractor channels when the weight search finds an alternative route.

Gating also scales cleanly with grid count. The same gated-mapper architecture tested on 3-, 4-, and 5-grid conjunctions achieves 100% success at 3-grid with a median convergence of 34 generations, 2.3× faster than the ungated three-grid composition baseline (78 gen); 100% at 4-grid in 37 gen, matching the four-grid composition baseline while the four-grid dynamic-function baseline fails at 87%; and 96.6% at 5-grid in 489 gen, with the single failure ($f = 0.989$) exhibiting correct gate patterns (all functional gates above 0.98 and the suppressed gate at 0.03), placing it below the strict $f \geq 0.99$ threshold used for 5-grid analyses. Gating therefore delivers reliable selective-routing behavior up through the decisive 5-grid threshold; what it does not yet deliver is task-agnostic routing. Gate weights remain task-specific (re-evolved per compound task), so the architectural bias addresses the indiscriminate-routing problem without resolving the generalization problem.

Pushing the conjunction to six grids ($\text{XOR}(a,b) \wedge \text{AND}(c,d) \wedge \text{OR}(e,f) \wedge \text{NAND}(g,h) \wedge \text{NOR}(i,j) \wedge \text{XNOR}(k,l)$, 12 inputs, 4096-row truth table) reaches the first scale at which the augmented pipeline itself becomes unreliable. The all-zeros predictor scores $f = 0.9912$ on this truth table, so the success threshold tightens to $f \geq 0.995$ to keep a margin against majority-class output; under that threshold the augmented mapper solves only 1 of 29 seeds in 2000 generations, with the other 28 plateauing at $f \in [0.992, 0.993]$. Spot-checked seeds at the plateau predict negative on every positive pattern (true-positive rate 0), the same failure mode that 5-grid monolithic baselines exhibit. Gating substantially mitigates this collapse: with the same six frozen specialists and the same threshold, the gated mapper solves 21 of 29 seeds (72%) at median 362 generations, a 21× improvement under identical conditions, with multiple viable gate signatures across seeds (some suppress NOR and XNOR, others suppress OR and NAND). At 5-grid composition rescues the monolithic baseline; at 6-grid gating rescues composition. The complexity threshold therefore has two stages within the Boolean regime, and the prototype's reach extends one grid further than the augmented pipeline alone would suggest.

These results provide the first empirical evidence that emergent compositional behavior is possible within the GEENNS framework. The mappers were not told which inputs to route to which grids; they discovered the correct routing through evolutionary pressure. The composition was not programmed; it emerged from evolved connection weights. This is emergence in a precise, limited sense: the compositional strategy was discovered by the evolutionary process, not designed by the experimenter. The limitation is equally clear: the emergence is task-specific, not general-purpose. Individual mappers learn weight patterns that solve a particular compound task rather than transferable compositional strategies, yet the population-level analysis developed later in this section shows the transferable structure is present in the wider population even when it is absent in any single mapper.

7.6.3 Scaling Behavior: Composition vs. Baselines

Table 7.7 and Figure 7.6 consolidate the success rates and convergence trajectories across the five grid scales tested in the prototype, comparing the compositional pipeline against three baseline conditions: dynamic per-node function selection (a monolithic network whose CPPN assigns activation functions per node, from work beyond this thesis; included as diagnostic baselines, not thesis contributions), mono-tanh, and mono-sin. The 6-grid Boolean probe (§7.6.2) uses augmented-versus-gated composition only, because the monolithic baselines had already collapsed at 5-grid.

Table 7.7: Success rates across 1–5 grid scales. Composition maintains 100% at every scale; convergence speed improves from 78 gen (3-grid) to 40 gen (4-grid), then rises to 281 gen at 5-grid as the conjunctive search depth grows. Dynamic functions match at 2- and 3-grid, degrade to 87% at 4-grid with an 8× convergence speed inversion (Mann-Whitney $p = 3.2 \times 10^{-8}$, $r = 0.87$), and collapse to 0% at 5-grid (Fisher's exact $p < 10^{-17}$ vs. composition). Uniform-activation baselines collapse at 3+ grids. Each cell shows success rate (with seeds converged in parentheses) above and median convergence generation below; "—" indicates no successful seeds.

Scale	Composition	Dynamic	Mono-tanh	Mono-sin
1-grid	100% (30/30) 1 gen	100% (30/30) 1 gen	37% (11/30) 95 gen	100% (30/30) 1 gen
2-grid	100% (30/30) 9 gen	100% (30/30) 3 gen	80% (24/30) 178 gen	100% (30/30) 82 gen
3-grid	100%[§] (29/30) 78 gen	100% (30/30) 22 gen	3% (1/30) 359 gen	0% (0/30) —
4-grid	100%[§] (29/29) 40 gen	87% (26/30) 321 gen	27% (8/30) 437 gen	0% (0/30) —
5-grid	100%[§] (29/29) 281 gen	0% (0/30) —	0% (0/30) —	0% (0/30) —

[§]One seed excluded: OR specialist grid did not converge (fitness 0.865). 5-grid uses $\text{XOR}(a, b) \wedge \text{AND}(c, d) \wedge \text{OR}(e, f) \wedge \text{NAND}(g, h) \wedge \text{NOR}(i, j)$ (10-input, 1024-row truth table). 1–4 grid rows use success threshold $f \geq 0.95$; the 5-grid row uses $f \geq 0.99$ because the 18/1024 positive-pattern ratio makes an all-zeros predictor reach $f = 0.982$ from class imbalance alone.

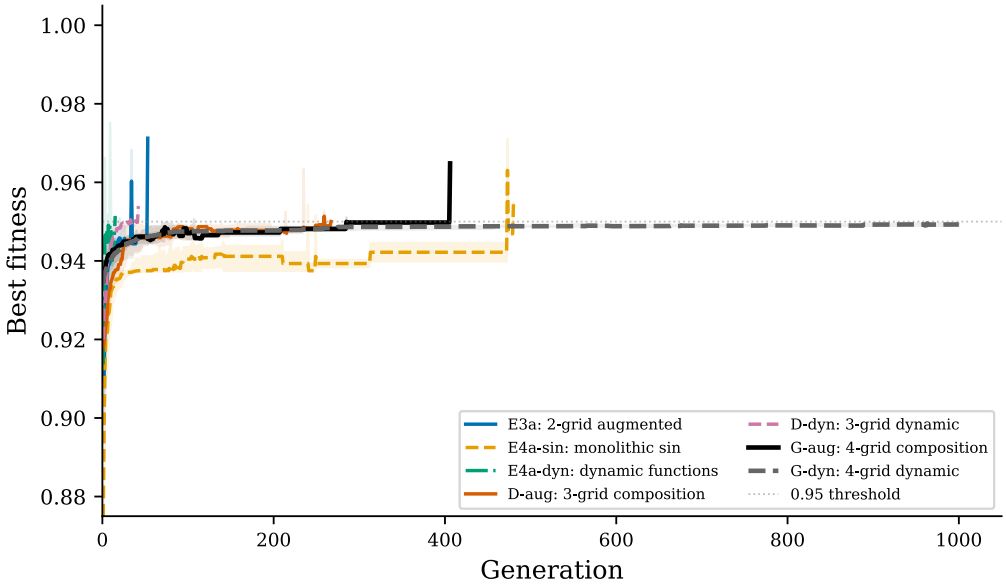

Figure 7.6: Convergence curves (median ± IQR) for key conditions across 2-, 3-, and 4-grid scales. Two-grid routing converges in 9 generations vs. the 2-grid mono-sin baseline in 82, a 9× speedup from frozen grid outputs. At 3-grid, the dynamic baseline converges faster (median 22 gen) than composition (78 gen). At 4-grid, the speed relationship inverts: four-grid composition converges in median 40 gen vs. the four-grid dynamic baseline in 321 gen, an 8× speed advantage (Mann-Whitney $p = 3.2 \times 10^{-8}$) that defines the complexity threshold. The 0.95 fitness threshold is shown as a dashed line.

Three patterns emerge from the scaling analysis. First, composition is *reliably* scale-invariant: 100% success at every scale tested, from 1-grid through 5-grid. Median convergence generations trace a non-monotonic curve (78 at 3-grid, 40 at 4-grid, 281 at 5-grid): the 4-grid speedup reflects stronger mapper signals from an extra grid output, while the 5-grid regression reflects the 4× truth-table expansion (256 → 1024 rows) and the strict $f \geq 0.99$ threshold required to exclude majority-class prediction. Second, there is a *monotonic degradation hierarchy* at 4-grid scale: composition (100%) > dynamic (87%) > mono-tanh (27%) > mono-sin (0%); at 5-grid the hierarchy compresses to composition (100%) > everything else (0%). Third, the *speed inversion is extreme*: at 3-grid, the dynamic-function baseline converges 3.5× faster than composition (22 gen vs. 78 gen); at 4-grid, composition converges 8.0× faster than the dynamic-function baseline (40 gen vs. 321 gen), a ∼28× swing in relative speed advantage, which at 5-grid becomes a reliability gap rather than a speed gap because the dynamic baseline no longer converges at all.

The mono-tanh recovery at 4-grid (3% at 3-grid → 27% at 4-grid) warrants explanation. NAND is linearly separable, giving the 4-gate truth table more exploitable structure than the 3-gate conjunction. The 8/30 solves are hard-won (median 437 gen, range 130–861) and statistically distinguishable from chance (Fisher's exact $p = 0.002$ against 0/30). This does not invalidate the degradation hierarchy; it shows that task structure interacts with activation function capability at every scale.

The scaling table provides the clearest evidence for a *confirmed* complexity threshold. At 2-grid scale, dynamic functions are strictly superior (100% at 3 gen vs. 100% at 9 gen). At 3-grid, dynamic functions still match on success rate but composition closes the speed gap. At 4-grid, composition pulls ahead on both reliability and speed: 100% at 40 gen

vs. 87% at 321 gen. The success rate difference alone is not significant at conventional levels (Fisher's exact $p = 0.112$), but the convergence speed inversion is highly significant (Mann-Whitney $p = 3.2 \times 10^{-8}$, $r = 0.87$) and the four dynamic-baseline failures plateau at $f = 0.949$–0.950, indicating capacity limits rather than slow convergence. Increasing compound task complexity from 3 to 4 gates produces simultaneous reliability degradation and speed inversion. This establishes a complexity threshold beyond which modular architecture becomes both necessary and sufficient.

The 5-grid experiment sharpens this threshold to the point of decisiveness. Extending the conjunction to $\text{XOR}(a, b) \wedge \text{AND}(c, d) \wedge \text{OR}(e, f) \wedge \text{NAND}(g, h) \wedge \text{NOR}(i, j)$ (10 inputs, 1024 truth-table rows), composition retains 100% success (29/29, median 281 gen, mean fitness 0.992), while every monolithic baseline collapses to 0% (0/30 for dynamic functions, mono-tanh, and mono-sin alike). Only 18 of the 1,024 truth-table rows evaluate to positive (roughly 1.76%), so a predictor that outputs zero on every input already scores $f = 0.982$; we therefore tighten the success threshold to $f \geq 0.99$ for 5-grid analyses to force genuine learning above the class-imbalance floor. The four-grid gap of 87% versus 100% (marginal at $p = 0.112$) widens to 0% versus 100% at 5-grid for every composition-vs-baseline contrast. The dynamic baseline reaches a mean fitness of 0.987, which superficially looks like near-convergence, but recall the 0.982 floor that the constant-zero predictor already scores. To check whether the baseline learned anything beyond that trivial predictor, we re-evolved five of its seeds and inspected their per-pattern outputs: on the 18 positive patterns the output averages 0.27, below the 0.5 decision boundary, and the seeds correctly flag only about 1.4 of the 18 positives on average, a true-positive rate near 7.8% (the tanh variant scores worse at 5.6%). The 0.987 fitness is an artifact of class imbalance: the baseline reaches it by predicting negative on nearly every input, not by learning the compound function. At 5-grid the threshold is no longer marginal. Monolithic substrates do not merely converge more slowly; they do not converge at all. Whether this threshold generalizes beyond Boolean conjunctions, to domains where task structure differs from gate composition, remains the central open question.

A theoretical argument predicts this threshold. The monolithic dynamic-function substrate is a single EMR-HyperNEAT network whose CPPN must simultaneously discover topology, connection weights, and the correct spatial partitioning of activation functions, assigning `sin` to the region processing XOR and `tanh` to regions processing the linearly separable gates. As the compound task scales from 3 gates (6 inputs, 64-row truth table) to 4 gates (8 inputs, 256-row truth table), the search space for correct function-to-region mapping grows combinatorially while the actual computational difficulty of the sub-tasks does not change: NAND is linearly separable, like AND and OR. By contrast, the compositional pipeline's difficulty scales additively: one more frozen grid, one more mapper input. The CPPN's spatial partitioning problem is the bottleneck, not the task difficulty per se.

7.6.4 The Confounds

Three observations qualify the results. First, the dynamic function confound. At 3-grid scale, the dynamic function baseline, a monolithic EMR-HyperNEAT substrate with per-node activation function selection via CPPN output dimensions, achieves 100% at median 22 generations, *faster* than composition's 78 generations. The dynamic function baseline matches composition on solvability and beats it on speed at 3-grid scale. The central question is whether this advantage persists at higher complexity.

The crossover documented in §7.6.3 (3.5× dynamic advantage at 3-grid inverting to an 8.0× compositional advantage at 4-grid) is what resolves the confound. At $N = 30$, the four dynamic-baseline failures plateau at $f = 0.949$–0.950 (253/256 truth-table rows), indicating a capacity limit rather than slow convergence. The 13-percentage-point success-rate gap is not significant on its own (Fisher's exact $p = 0.112$), but the convergence-speed inversion is unambiguous, and the capacity-limit failure mode confirms that the inversion reflects substrate limits rather than slow search.

This constitutes a *complexity threshold*: the scale beyond which a monolithic CPPN can no longer reliably discover correct function-to-region partitioning, and compositional architecture becomes structurally necessary. The evidence is convergence speed, not success rate alone: the 8.0× speed advantage ($p < 10^{-7}$) and the capacity-limited plateaus in the four dynamic-baseline failures indicate a qualitative transition from "dynamic functions suffice" at 3-grid to "composition is both faster and more reliable" at 4-grid.

Second, a confound in the original noise control has a principled resolution. The initial attempt (70% with fixed-random distractor columns) was invalidated by determinism: because the injected values were constant per input pattern, a mapper could learn them as a 16-entry lookup table rather than ignore them. Two replacement conditions separate the two failure modes the original test conflated. A constant distractor column fixes success at 47%: evidence that extra uninformative input dimensions expand the effective search space and actively slow convergence, even when they carry no task-relevant signal. A resampled-noise condition recovers 100%: because the distractor values change per evaluation, no deterministic lookup exists, and the mapper is steered toward the stable grid outputs. The effect is analogous to input dropout as a regularizer; the interpretation is that resampled noise acts as an implicit pressure toward grid utilization, not as a generalization test in itself.

Third, connection-weight analysis in the frozen grid reuse experiment reveals that the unconstrained mapper distributes nearly uniform attention across grid-output columns, with correct-to-distractor weight ratios of 0.91–0.94×. This is the routing-indiscrimination phenomenon that gating addresses in §7.6.2: gating produces task-specific routing strategies (selective, inverted, or neutral depending on task configuration) rather than uniformly suppressing distractor channels.

7.6.5 Beyond Boolean: Continuous Gate Composition

The Boolean conjunction tests establish that composition scales where monolithic baselines collapse, but the task class is narrow: each gate is a discrete function on $\{0,1\}^2$, and the compound output is a single binary value. To probe whether the scaling story reflects an architectural property or an artifact of Boolean structure, the prototype includes continuous counterparts: XOR_c, AND_c, OR_c defined as smooth real-valued surfaces on $[0,1]^2$, each grid targeting its own regression objective, and the mapper composing their outputs into a continuous scalar target. Success is measured by $R^2 \geq 0.85$.

Under a multiplicative composition operator (the mapper combines grid outputs as a product, the continuous analog of logical conjunction), the complexity threshold shifts forward relative to the Boolean case. At 2-gate scale, composition alone reaches 100% (29/29; mean $R^2 = 0.860$, median 12 gen) while every monolithic baseline scores 0% (mean R^2 between 0.52 and 0.74, all below the 0.85 threshold). The contrast with the Boolean regime is sharp: at 2-grid Boolean scale the dynamic-function baseline matched composition (100%, median 3 gen), so composition there was optional rather than necessary.

In the continuous regime every monolithic baseline scores 0% at the *smallest* compound scale, so composition is already required, not optional. At 3-gate, composition drops to 55% (16/29 solved; mean $R^2 = 0.846$ across all seeds), and at 4-gate to 0% (0/29 reach the strict $R^2 \geq 0.85$ threshold; mean $R^2 = 0.824$). The 4-gate result is a near-miss rather than a collapse: composition seeds cluster between $R^2 = 0.80$ and 0.84, all monolithic baselines sit near zero (mono-tanh 0.152, mono-sin 0.022), and loosening the threshold to $R^2 \geq 0.80$ lifts composition to 93% (27/29) while baselines remain 0%.

The mechanism behind successful composition is not literal multiplication. If the mapper simply multiplied the four frozen-grid predictions, compounding sigmoid compression across the chain would cap performance at the multiplicative ceiling $\prod R_i^2 \approx 0.764$ at 4-gate; instead, every one of the 29 composition seeds at 3-gate and 4-gate exceeds this ceiling, by mean gaps of +0.029 and +0.059 respectively. Replaying the converged mappers across a uniform grid of gate-output values lets us ask what function the mapper actually computes. A degree-3 polynomial in the gate outputs accounts for ∼96.5% of its input–output variance at both scales (mean $R^2 = 0.965 \pm 0.007$ at 3-gate, 0.965 ± 0.000 at 4-gate), whereas a least-squares rescaled product captures only 90% at 3-gate and 87% at 4-gate. Even at 4-gate, where the true target is itself a degree-4 polynomial, extending the fit to degree-4 gains only +0.006 R^2 over degree-3. The mapper has settled on a smooth low-order approximation that caps how much per-gate sigmoid compression can compound: degree-3 truncation holds regardless of gate count, while literal multiplication would lose accuracy with every added gate. This adaptation is specific to the compositional architecture: none of the monolithic conditions exhibit it.

Swapping the multiplicative operator for an *additive mean* (the mapper averages grid outputs rather than multiplies them) erases the barrier entirely. Success returns to 100% at 3-gate (29/29, median 21 gen, versus the multiplicative 55%), holds at 4-gate (29/29, median 24 gen, versus the multiplicative 0%), and holds again at 5-gate (29/29, median 9 gen). The architecture, mappers, frozen grids, and training budget are otherwise unchanged; only the composition operator differs.

Additive monolithic baselines remain at 0% (mono-tanh 0/30, mono-sin 0/30, mono-dyn 0/30 at 4-gate, with mean $R^2 \approx 0.37$ in every case), so the operator change shifts the compositional pipeline from 0% to 100% without making the underlying compound function any more tractable for a monolithic substrate.

The reading is that the continuous scaling limit sits in the operator, not the substrate: products compound per-gate sigmoid-compression errors across grids, while averaging lets those errors cancel instead of accumulate. Pushing the population from 300 to 1,000, or the generation budget from 500 to 2,000, never pushes the multiplicative 3-gate rate beyond ≈55–62%, which rules out search-budget limits and points squarely at the operator. At the prototype's current scale the composition operator, not the architectural capacity of the grids or mappers, is the bottleneck beyond the Boolean domain. This reads as a retrospective clarification of the Boolean regime as well: Boolean conjunction fixes the operator at logical AND, which folds operator effects into the architecture and makes them invisible to ablation, so the Boolean experiments could not have surfaced operator dependence even in principle. Only by stepping outside Boolean structure does the operator become a free architectural choice, and only then does its role separate from the substrate's.

Three additional candidates beyond multiplicative and additive-mean fail uniformly at continuous gate composition: $\max_i g_i$, $\min_i g_i$, and a soft-max-weighted sum with $\beta = 5$ each solve 0 of 29 seeds at both 3-gate and 4-gate (174 runs total, mean R^2 in the

range 0.48–0.67). The operator landscape is narrower than its size suggests: among the five operators tested, only additive-mean reliably scales beyond 3-gate. The smooth sin-hidden mapper is plausibly mismatched to operators with kinks at switchover points, although optimization difficulty cannot be ruled out as a contributing factor.

Pushed further, additive composition shows no observed failure point. Re-evolving three additional 2-input continuous specialists (continuous XNOR, IMPLY, NIMPLY, all with sin activation) and assembling 6-, 7-, and 8-gate compounds, the pipeline solves 29/29 seeds at every gate count, with median convergence growing approximately linearly ($\Delta \approx 4.3$ generations per gate added; from 9 generations at 5-gate to 22 at 8-gate). Mean R^2 rises from 0.888 at 5-gate to 0.905 at 8-gate. Through eight continuous gates the architecture, frozen grids, and mapper population reliably scale; the only continuous bottleneck observed remains on the multiplicative side.

For GEENNS, the original vision posits frozen specialist grids and an evolved mapper that composes their outputs; the mapper's compositional *operator* has been an implementation detail rather than an explicit design axis. These results suggest that operator choice (multiplicative, additive, mixture-of-experts gating, or task-dependent) is itself an explicit evolutionary variable whose selection co-determines what compositions are representable.

7.6.6 Synthesis: What the Prototype Answers

The seven prototype questions of §7.6 find the following answers in the Boolean conjunction tests and the continuous-gate extension.

Feasibility, composition, routing, and grid utilization (PQ1–PQ4) are answered affirmatively. EMR-HyperNEAT specialist grids converge reliably across all five tested Boolean gates, providing frozen modules suitable for composition (PQ1; §7.6.2). Evolved mappers learn to compose pre-solved grid outputs into compound solutions, with the routing discovered by evolution rather than designed (PQ2; §7.6.2, §7.3.3). At small grid counts the compositional pipeline delivers a speed advantage over the monolithic baseline; beyond the complexity threshold the speed advantage gives way to a capability advantage (PQ3; §7.6.3). The mapper genuinely uses grid outputs rather than bypassing them, although weight magnitude and ablation impact tell complementary stories (PQ4; §7.6.2, §7.6.4).

Generalization and reuse (PQ5–PQ6) succeed with architectural support, not at the individual mapper level. Individual mappers do not zero-shot unseen compositions, but the source-evolved populations contain transferable structure; generalization is therefore a population-level property of the evolved search, not an individual-mapper property, and the open problem is extracting this signal into a single general mapper (PQ5; §7.6.7). The same frozen grids support mapper evolution for different compound tasks (PQ6; §7.6.2). Unconstrained mappers route indiscriminately, but architectural gating elicits selective routing when task structure admits it, though gate weights remain task-specific.

Scaling (PQ7) is the decisive result. Compositional architecture maintains reliable convergence at every tested scale while monolithic baselines degrade monotonically and collapse at the largest scale. The complexity threshold manifests at 4-grid as a

convergence-speed inversion and at 5-grid as a reliability gap. Continuous-gate experiments relocate the bottleneck from substrate capacity to the composition operator itself (§7.6.3, §7.6.5).

7.6.7 Limitations and Open Questions

The seven prototype questions answered above leave one deeper question open: whether the *individual mapper* learns compositional reasoning in the sense the GEENNS vision requires, and the answer given by the compositional generalization test alone is "no." The single best individual per target reaches fitness 0.53–0.80 across unseen compositions, with the population mean near 0.53, well below the 0.95 convergence threshold. Individual mappers therefore learn task-specific weight patterns, not general compositional strategies. But this is not the whole answer.

Warm-start transfer experiments reshape this picture without overturning it. Reseeding the mapper population with individuals evolved on a source compound task yields 45–79× convergence speedups on related targets at 3-grid scale, and 223–351× at 5-grid. The source-evolved populations contain roughly 0.3% latent solvers of unseen targets (individuals that, without any retraining, already satisfy the target objective), whereas random-initialization populations scaled up to pop 3000 contain fewer than 0.01%. Across source and target solver sets the Jaccard overlap is 1.0 at the median, 0.88 on average: the same individuals solve both targets in the typical pair, with a small fraction of source-only or target-only solvers in a minority of cases. The transferred signal is therefore not two different solutions of equivalent quality but, overwhelmingly, a single representation that encodes structure common to both compositions. The implication is that generalization in this architecture is a *population-level* property rather than an individual-mapper property: mappers do not zero-shot unseen compositions, but evolved populations contain transferable compositional structure that new search can exploit.

The same-grid speedups understate one constraint. Substituting a single specialist at 5-grid Boolean scale (NOR→XNOR, the missing oscillatory primitive) reduces the warm-start advantage to a 2.06× ratio-of-medians (cold 828 gen, warm 402 gen) and lifts no seed beyond the cold-start solve rate (22/29 either way). Only 4/29 source-evolved populations contain a zero-shot solver of the substituted target at the strict $f \geq 0.99$ threshold, although mean zero-shot fitness remains high ($\bar{f} = 0.984$). Population-level transfer at 5-grid Boolean is therefore conditional: it holds under input-pair permutation, where the source compositional structure is reused with relabeled coordinates, but it weakens under gate-set substitution, where the source-evolved distribution is partly tuned to the specific gate set. Under the additive operator the same mechanism transfers cleanly: at 5-gate continuous scale with $\text{NOR}_c \rightarrow \text{XNOR}_c$, 12 of 14 source-evolved populations contain a zero-shot solver of the unseen target (mean target $R^2 = 0.896$); at 6-gate ($\text{NOR}_c \rightarrow \text{IMPLY}_c$) the zero-shot rate drops to 59%; at 7-gate ($\text{NOR}_c \rightarrow \text{NIMPLY}_c$) it returns to 86%. The non-monotonic trajectory tracks the structural distance between the original and substituted gates rather than compound size, suggesting that what the source population encodes is partly compositional and partly distributional, a refinement of, not a contradiction to, the population-level transfer story.

One plausible reading of this result, which the prototype motivates but does not prove, is that compositional generalization in this architecture may be accessible through an *exposure-and-extraction* dynamic rather than a per-individual learning dynamic. On this reading, the 0.3% of source-evolved mappers identified by warm-start as latent solvers

of the unseen target *are* the compositional signal, and the open problem is how to bias evolutionary search toward that fraction of the population or how to distill it into a single general mapper. Whether this reading holds at larger grid counts, across task families, or under different source tasks is untested; what the prototype establishes is only that the signal is present, reliably recovered by warm-start, and not present in random initialization at ten-times the search budget.

Bottom line. Composition is the only condition that remains reliable as grid count rises (Table 7.7), and the gap widens from marginal at 4-grid to decisive at 5-grid, where every monolithic baseline collapses to 0%. Modular reuse is validated (PQ6); selective routing scales through gated mappers at 3-, 4-, and 5-grid (100%/100%/96.6%); and continuous-gate experiments relocate the scaling limit from architecture to composition operator. Warm-start transfer shows generalization is possible at the population level (∼0.3% latent solvers) even though individual mappers fail to zero-shot unseen compositions. What remains open is no longer "does modular composition scale" but a cluster: scaling beyond 5 grids and beyond Boolean tasks, designing composition operators for continuous domains, and condensing the population-level transfer signal into a single general-purpose mapper.

7.7 Chapter Summary

This chapter established three things.

First, GEENNS's design draws on three biological and computational principles (columnar homogeneity with functional diversity, evolutionary architecture search, and emergent modularity under compositional task pressure) and specifies an architecture of grids as domain specialists and mappers as compositional coordinators.

Second, three barriers block the path from substrate to compositional intelligence: computational (quadtree bottleneck, Chapters 5–6), expressiveness (uniform activation functions, §8.4), and multi-task (no task-specific mechanism, Chapter 4 and §8.4). Removing these barriers is the thesis's contribution. A prototype of approximately 3,100 runs validates the modular architecture up to 5-grid scale: compositional pipelines retain 100% success on XOR ∧ AND ∧ OR ∧ NAND ∧ NOR while every monolithic baseline (mono-tanh, mono-sin, and dynamic per-node function selection alike) scores 0%. The 4-grid convergence speed inversion first signaled that the complexity threshold widens at 5-grid into a reliability gap, converting an arguable efficiency advantage into a decisive capability advantage. Gated selective routing scales through 3-, 4-, 5-, and 6-grid Boolean conjunctions (100%/100%/96.6%/72% success), rescuing the augmented pipeline at the 6-grid scale where it falls to 1/29; the complexity threshold therefore has a two-stage structure within the Boolean regime. Continuous-gate experiments relocate the scaling limit from substrate capacity to the *composition operator*: multiplicative composition fails at 4-gate while an additive mean holds at 100% through at least 8 continuous gates with approximately linear growth in convergence cost. Warm-start transfer reframes generalization as a population-level property: source-evolved populations contain roughly 0.3% latent solvers of unseen targets, against fewer than 0.01% in random-initialization populations even at pop 3000. Three open problems follow directly: (a) scaling compositional reliability beyond five grids and beyond Boolean conjunctions; (b) moving past the multiplicative ceiling in continuous composition, where the operator rather than the architecture dictates the limit; and (c) extracting a single general-purpose mapper from the population-level transfer signal. The full formal specification (multi-level CPPN hi-

erarchy, developmental pipeline, plasticity mechanisms, and mathematical formalization) is preserved in Appendix B as a design target for future implementation.

Third, the prerequisite chain itself was a discovery: each barrier had to be solved before the next could be attempted, revealing the substrate problem as thesis-scale in its own right. With the prototype evidence in place, the thesis moves GEENNS from a theoretical design to a testable substrate-and-mapper pipeline whose remaining open questions are now specific rather than diffuse: the composition operator in continuous domains, the extraction of population-level transfer into single mappers, and scaling beyond five grids and Boolean conjunctions.

Appendix B

GEENNS Formal Specification

> Should philosophy guide experiments, or should experiments guide philosophy?
>
> Liu Cixin

This appendix contains the detailed formal specification of the GEENNS algorithm, including the multi-level CPPN hierarchy, the developmental pipeline, plasticity mechanisms, the continual learning comparison, detailed architecture figures, and mathematical formalization. All content is a *design specification*; no implementation or validation exists beyond the prototype evidence in Section 7.6. The condensed architectural overview appears in Chapter 7.

Implementation Status. This appendix specifies the GEENNS algorithm as designed but *not yet fully implemented*. Only the prototype pipeline (Chapter 7, approximately 3,100 runs spanning Boolean compounds up to 6-grid and continuous additive compounds through 8 gates) has been validated, and only using a simplified augmented-input mapper over a two-layer EMR-HyperNEAT substrate. The three-type mapper formalism of Definition B.2 (with explicit inter-grid routing), the plasticity mechanisms, and the multi-level CPPN hierarchy (Section B.4) all remain theoretical. Per-node activation and aggregation function assignments described throughout this appendix are design targets for future work, not implemented thesis contributions; the feasibility of per-node function evolution is explored in companion work beyond this thesis (§8.4). This specification is a design document for future implementation, not a description of working software.

B.1 Substrate Evolution Landscape and CoDeepNEAT Comparison

See Section 7.4.3 for the condensed overview and Section 7.6 for the experimental validation.

GEENNS grids are spatially organized substrates with columnar structure, CPPN-generated connectivity, and per-node function diversity. The substrate evolution method must (a) scale to large networks while preserving geometric regularities, (b) support per-node features at low additional cost, and (c) produce substrates that can be frozen and

reused across tasks. Table B.1 compares the most relevant alternatives to explain why EMR-HyperNEAT is the necessary foundation.

DeepNEAT and CoDeepNEAT. Miikkulainen et al. [67] introduced DeepNEAT, which evolves layer-level topology, and CoDeepNEAT, which co-evolves two populations: *modules* (small network subcomponents) and *blueprints* (directed graphs specifying how modules connect).

CoDeepNEAT as closest conceptual prior art. CoDeepNEAT's blueprint/module split is the closest prior art to GEENNS's mapper/grid split: both separate *what computes* from *how components are assembled*. Two differences explain why CoDeepNEAT cannot serve as the substrate engine for GEENNS:

1. **Direct encoding cannot produce spatial substrates.** CoDeepNEAT modules are small network graphs where each connection is an individual gene. This means modules cannot exploit geometric regularities (symmetry, periodicity, spatial locality) that CPPNs provide through indirect encoding. GEENNS grids are spatially organized substrates where node placement, connectivity patterns, and per-node functions are all generated from spatial coordinates. A compact CPPN genome produces a large grid with systematic internal structure (repeated motifs, symmetric connectivity, gradient-based density variation) that would require a correspondingly large direct-encoding genome to specify explicitly. For compositional architectures built on columnar spatial organization, indirect encoding is not optional: it is structurally necessary.

2. **Co-evolving modules drift.** CoDeepNEAT modules are not frozen; they co-evolve alongside blueprints, so module capabilities shift continuously as blueprints change. A blueprint that learned to use Module Species A for feature extraction may find that A has drifted toward a different function by the next generation. GEENNS's freeze-then-route design eliminates this drift: grids are trained, frozen, and then mappers evolve against stable specialists.

Why EMR-HyperNEAT specifically. ES-HyperNEAT already provides indirect encoding with adaptive substrate discovery: the right foundation. But its sequential quadtree makes it computationally infeasible for GEENNS (Section 7.5.1). EMR-HyperNEAT solves this with the lazy-to-eager reformulation described in Section 7.2 (detailed in Appendix A.2.5), but also provides a second capability: because CPPN evaluation is now a batch matrix operation, adding per-node output dimensions (activation function index, aggregation function index, receptor densities) costs almost nothing. ES-HyperNEAT would require separate sequential quadtree passes for each additional feature, making per-node function evolution infeasible. EMR-HyperNEAT makes it a free byproduct. GEENNS grids need per-node function diversity (the expressiveness barrier, Section 7.5.2), and EMR-HyperNEAT is currently the only substrate evolution method that provides this at practical cost.

Table B.1: Comparison of substrate/network evolution approaches along dimensions relevant to GEENNS requirements. "Spatial" indicates whether the encoding preserves geometric regularities. "Per-node" indicates whether the method supports evolving per-node activation or aggregation functions at practical cost (a future direction enabled by EMR-HyperNEAT; see §8.4).

Algorithm	**Encoding**	**What It Scales**	**Spatial**	**Per-Node**	**GEENNS Suitability**
NEAT	Direct	Topology (∼1K nodes)	No	No	Foundation only; no substrates
HyperNEAT	Indirect (CPPN)	Substrate size	Yes	No	Spatial grids yes; fixed resolution
ES-HyperNEAT	Indirect (CPPN)	Adaptive density	Yes	No	Right approach; quadtree bottleneck
DeepNEAT	Direct (layers)	Network depth	No	No	Wrong abstraction for grids
CoDeepNEAT	Direct (blueprint + module)	Modular networks	No	No	Closest concept; wrong encoding
EMR-HyperNEAT	**Indirect (CPPN)**	**Tensor-vectorizable substrate**	**Yes**	**Yes**	**Built for GEENNS**

Co-evolution as future work. CoDeepNEAT's blueprint/module co-evolution raises a natural question for GEENNS's future: should mappers and grids co-evolve rather than following the current freeze-then-route pipeline? Co-evolving external parameters alongside NEAT populations would face a population alignment problem: NEAT's selection mechanism reorders population indices between generations, breaking the mapping between genomes and their co-evolved parameters without any explicit error signal. GEENNS's freeze-then-route approach sidesteps this entirely by decoupling the two evolutionary processes. A hybrid approach (co-evolving mappers with grids under constrained plasticity) is worth exploring in future work, but will require solving the alignment problem more reliably than current methods allow. The approach would likely differ from CoDeepNEAT's continuous co-evolution; a staged pipeline (evolve grids → partially freeze → co-refine mappers and grid plasticity) may be more tractable.

B.2 Why Evolution Determines Architecture

Conventional architectures (CNNs, Transformers) are hand-designed for specific hardware families and problem domains. GEENNS's substrate must be jointly optimized for hardware constraints (memory bandwidth, parallelism granularity, communication topology) *and* the target task distribution (required computational primitives, routing patterns). The joint design space (column count, layer count, node density, connectivity patterns at three spatial scales, and per-node activation and aggregation functions explored in companion work, not a thesis contribution) is combinatorially large and hardware-dependent. Hand-crafting a substrate for every hardware–task combination is intractable (Figure B.1).

Hornik et al. [41] showed that a feedforward network with a single hidden layer of sufficient width and non-polynomial activation functions can approximate any continuous function on compact subsets of $\mathbb{R}^n$ (see also Cybenko [21] for the sigmoidal case). The theorem guarantees *existence* of a solution but not *discovery*: in fixed architectures, gradient descent adjusts weights but cannot alter topology. NEAT and its successors evolve topology alongside weights, searching the space of network structures. GEENNS extends this to hierarchical structures: evolution searches the space of columnar architectures for a substrate suited to both the target task distribution and the available hardware.

The universal approximation result justifies the search by guaranteeing the target space is not empty: a sufficiently expressive network exists for any continuous target function. It does *not* guarantee that evolution finds it, that the required network size is tractable, or that evolution outperforms gradient-based neural architecture search. Whether evolutionary search navigates this space for compositional substrates is an empirical question addressed throughout this thesis.

B.3 Architectural Detail: From Network to Node

This section decomposes the GEENNS architecture top-down, from the full multi-grid network to the per-node computation that unfolds at each propagation step. Figure B.2 locates the mappers relative to the shared columnar network; Figure B.3 zooms into a single grid to expose the four CPPN levels that govern intra-grid connectivity; Figure B.4 traces a concrete inject–propagate–extract cycle; Figure B.5 resolves the finest level of structural detail.

Full architecture and mapper placement. The outer frame of GEENNS is a columnar network flanked by task-specific mappers. Input mappers decompose a task signal into injection patterns for each grid; output mappers compose the grids' responses back into a task result. The columnar network itself is shared: all tasks route through the same underlying substrate, and only the mapper populations differ per task.

Intra-grid structure and CPPN hierarchy. Each grid in the architecture is itself a nested structure. Columns contain minicolumns, and each minicolumn is a stack of DES-HyperNEAT substrate layers. Four CPPNs govern connectivity at progressively finer scales, from the outermost inter-column wiring down to the per-layer substrate. Figure B.3 makes the hierarchy explicit by expanding a single column.

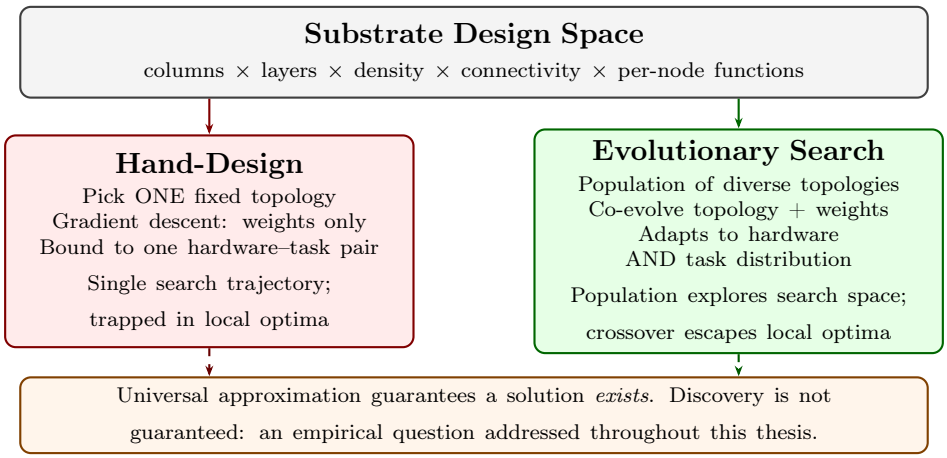

Figure B.1: Hand-designed versus evolutionary architecture search. The substrate design space spans column count, layer count, node density, connectivity patterns at three spatial scales, and per-node functions. Hand-design selects a single fixed topology; evolutionary search co-evolves topology and weights simultaneously. Universal approximation guarantees a solution exists but not that any search process finds it.

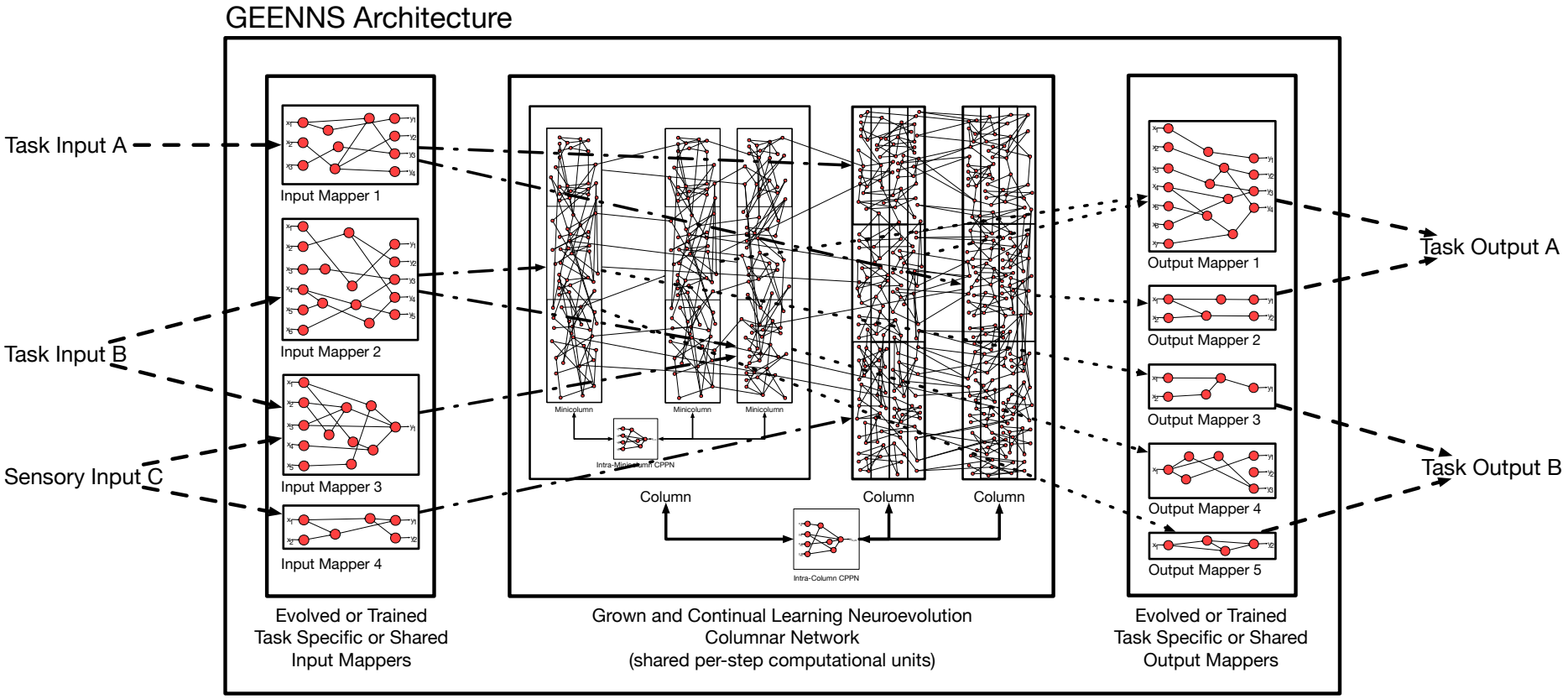

Figure B.2: Full GEENNS architecture. Input mappers (left) receive task and sensory signals and route them into the columnar network (center), which consists of three columns each containing multiple minicolumns with evolved internal connectivity. An Intra-Minicolumn CPPN governs within-column wiring; an Intra-Column CPPN governs between-column wiring. Output mappers (right) compose column responses into task-specific results. The columnar network is shared across all tasks; only mapper configurations change per task.

Figure B.3: GEENNS intra-column architecture. Three columns (outer rectangles) each contain multiple minicolumns, linked by an Intra-Column CPPN. The first column is expanded to show its internal structure: each minicolumn contains multiple layers, where each layer is a DES-HyperNEAT substrate with its own CPPN. Four levels of CPPNs govern connectivity at different spatial scales.

Grid computation in action. The structural picture becomes concrete when we trace a single forward pass. Consider a small grid of 3 columns × 2 layers × 4 nodes (24 nodes in total). Injection happens at $t = 0$: the input mapper writes a signal vector into the grid's designated input nodes (typically the top-layer nodes of selected columns). At each propagation step $t \to t{+}1$, every node aggregates its weighted inputs through its evolved aggregation function (one of `sum`, `min`, `max`, `mean`), adds its bias, and applies its evolved activation function (drawn from `sin`, `tanh`, `sigmoid`, and related families). When neuromodulation is active, the activation output is also scaled by the gain factor $\alpha_i(\mathbf{n}_t) = \mathbf{r}_i \cdot \mathbf{n}_t$: the dot product of node v_i's receptor density $\mathbf{r}_i$ with the task's neurotransmitter vector $\mathbf{n}_t$. After T propagation steps (typically 2–5 for Boolean tasks), the output mapper reads designated output nodes and produces the task result. Figure B.4 shows the complete cycle.

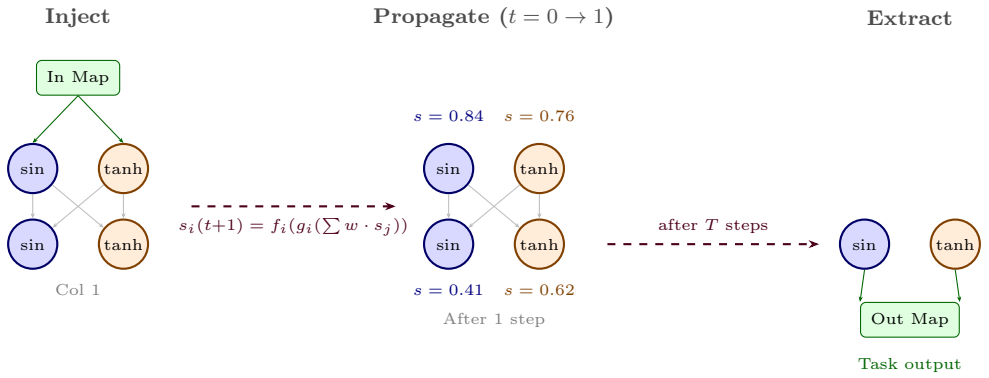

Figure B.4: Grid computation walkthrough. Left: the input mapper injects signals into designated input nodes (top layer). Center: after one propagation step, each node aggregates its weighted inputs and applies its per-node activation function (sin in blue, tanh in orange), producing updated state values. Right: after T propagation steps, the output mapper reads designated output nodes. Under neuromodulation, each node's activation is also scaled by a task-specific gain factor. Per-node function selection is explored further in companion work beyond this thesis.

Minicolumn-level detail. At the finest granularity, a minicolumn resolves into a vertical stack of DES-HyperNEAT substrate layers. Each layer carries its own CPPN for within-layer connectivity; Intra-Layer CPPNs link adjacent layers; and a Minicolumn CPPN defines the structural template shared across all minicolumns of a column. This is where the canonical microcircuit of Definition B.1 takes physical form.

B.4 Multi-Level CPPN Connectivity

GEENNS requires connectivity at multiple spatial scales, each generated by a dedicated set of CPPNs. Table B.2 specifies the complete CPPN hierarchy.

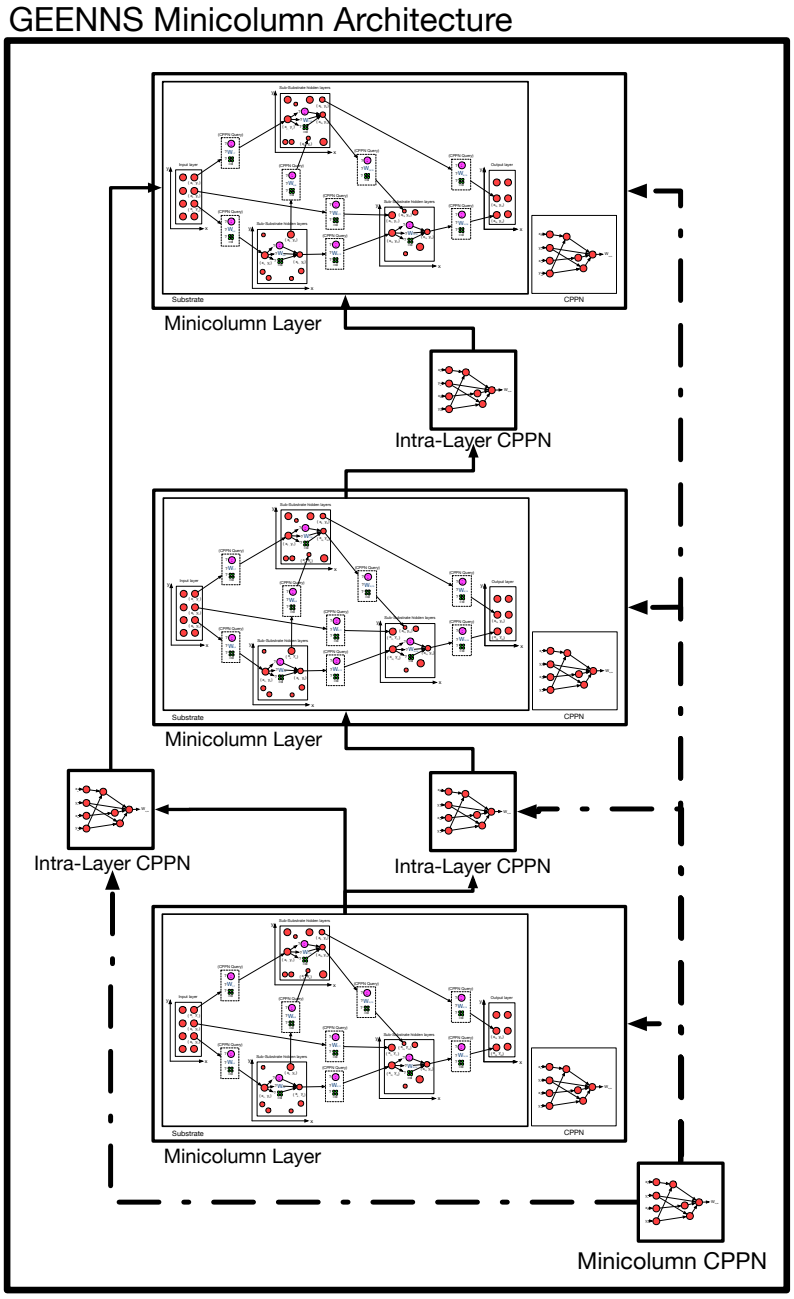

Figure B.5: GEENNS minicolumn architecture. Three minicolumn layers are stacked vertically, each containing a full DES-HyperNEAT substrate (with its own CPPN for intra-layer connectivity). Intra-Layer CPPNs generate the connectivity patterns linking adjacent layers. A Minicolumn CPPN governs the overall structural template shared across all minicolumns within a column.

Table B.2: CPPN hierarchy for GEENNS connectivity. "Shared across" indicates which structural elements use the same CPPN; "Scope" indicates what the CPPN connects. For k grids and T tasks, the total CPPN count is $4 + k(k-1)/2 + 2T$.

CPPN type	**Shared across**	**Scope**	**Input**	**Output**
Per-layer	All layers of same type	Within DES-HyperNEAT layer	4D coords	weight, prob
Intra-layer	All minicolumns	Between layers in minicolumn	4D coords	weight, prob
Intra-minicolumn	All columns	Between minicolumns	4D coords	weight, prob
Intra-column	All grids	Between columns	4D coords	weight, prob
Inter-grid	Per grid pair	Between grids ($k(k-1)/2$)	4D coords	weight, prob
Input mapper	Per task	Task input $\rightarrow$ grid inputs	varies	varies
Output mapper	Per task	Grid outputs $\rightarrow$ task output	varies	varies

The multi-level CPPN architecture is the primary source of the computational bottleneck identified in Section 7.5. With 4 base CPPNs for intra-grid connectivity at different scales, $k(k-1)/2$ inter-grid CPPNs, and 2 CPPNs per task (input and output mappers), a system with k grids and T tasks requires $4 + k(k-1)/2 + 2T$ CPPNs, each requiring its own hyperparameter optimization. For the prototype's 3-grid, 3-task configuration: $4+3+6 = 13$ CPPNs. For a full 10-grid, 20-task system: $4+45+40 = 89$ CPPNs. Part II of this thesis addresses the per-CPPN cost; Part III removes the computational bottleneck and introduces the substrate architecture that makes function-level expressiveness feasible as future work.

Feasibility envelope. The CPPN count translates directly into compute. Table B.3 extrapolates the wall-clock cost of a (k=5, T=10) sweep under XOR-scale per-trial assumptions; Section 8.3.1 expands the assumption stack.

Table B.3: Speculative wall-clock envelope (illustrative only) for a GEENNS hyperparameter sweep at a modest (k=5, T=10) scale: 34 CPPNs (formula above) at 2,013 TPE trials each (the budget inherited from the MNIST optimization of Chapter 3) for ∼68,000 configurations. Baseline extrapolates XOR-scale ES-HyperNEAT per-trial timings; the pruner savings (41.6%) are measured on MNIST (Chapter 3) and assumed to transfer; the GPU row applies the XOR-at-depth-7 EMR speedup. Measured EMR speedups vary with task, depth, and population: 5.5× on financial regression (depth 6, pop 100), 12–34× per-generation on XOR (depths 5–7, pop 1000), and a cumulative ratio of ∼33× over a 30-generation XOR run at depth 7 (with an asymptotic limit near ∼34× as JIT overhead amortizes; Chapter 6). The projections are favorable but speculative: the arithmetic stacks three assumptions (XOR-scale per-trial cost, 41.6% pruner transfer to GEENNS-scale problems, and the depth-7 EMR speedup applying uniformly across 34 CPPNs of varying purpose); each is defensible individually, but their composition is not validated end-to-end. Harder tasks increase per-trial cost super-linearly, so these figures are lower bounds on wall-clock, not predictions. §8.3.1 expands the caveats.

Configuration	**Est. Wall-Clock**	**Feasible?**
Baseline: quadtree, CPU	∼250 CPU-days	No
+ pruner (41.6% savings, MNIST-trained)	∼146 CPU-days	Marginal
+ EMR-HyperNEAT on GPU	∼4.9 GPU-days	**Yes**

B.5 The Developmental Process

GEENNS adopts the Phylogenesis–Ontogenesis–Epigenesis (POE) framework from evolvable hardware [84], decomposing the system's adaptation into three timescales:

Phylogenesis (evolutionary timescale). The genome, comprising NEAT-encoded microcircuit templates and CPPN-encoded connectivity patterns, evolves across generations. This is the mechanism by which GEENNS discovers general-purpose computational substrates. Phylogenesis operates at the population level: selection, crossover, and mutation produce new genome variants; fitness evaluation determines which variants survive.

Ontogenesis (developmental timescale). A single genome unfolds into a functional substrate through a five-stage developmental process (Figure B.6):

1. **Node generation.** Create $W \times H \times D$ nodes organized into W columns, each with H layers of D nodes, using the canonical microcircuit template Γ_{micro}.
2. **Layer organization.** Assign layer-specific properties (type, density) based on Γ_{micro}. Normalize node coordinates to the $[-1, 1]$ range for CPPN queries.
3. **Connection formation.** Query each CPPN in the hierarchy (Table B.2) at all relevant node-pair coordinates. Retain connections where the existence probability $|p_{ij}| > \theta_{\text{conn}}$. Assign per-node activation and aggregation functions from CPPN outputs.
4. **Refinement.** Prune connections with $|w_{ij}| < \theta_{\text{prune}}$. Apply variance-based filtering as in EMR-HyperNEAT (Chapter 6). Strengthen surviving connections through normalization.
5. **Maturation.** Initialize the receptor density matrix $\mathbf{R}$ for neuromodulation. Set initial plasticity parameters (learning rates, homeostatic targets). The grid is now operational.

This function is deterministic: the same genome always produces the same grid.

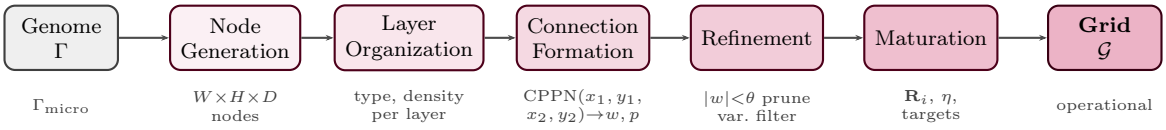

Figure B.6: Five-stage developmental pipeline. A genome Γ is transformed into a functional grid $\mathcal{G}$ through deterministic stages: node generation using the canonical microcircuit Γ_{micro}, layer organization assigning type and density, connection formation via CPPN queries over node-pair coordinates, refinement through pruning ($|w| < \theta$) and variance filtering, and maturation initializing the receptor density matrix $\mathbf{R}$ and plasticity parameters.

Epigenesis (operational timescale). During operation, plasticity mechanisms modify the substrate's behavior without altering its genome. GEENNS proposes four plasticity types, described in the following subsection.

B.6 Plasticity Mechanisms

GEENNS proposes four forms of plasticity operating during the epigenetic (operational) timescale. We distinguish which have experimental support and which remain theoretical.

1. **Hebbian plasticity.** Correlation-based strengthening and weakening of connections [37] ("cells that fire together wire together"). Connection weight w_{ij} is updated according to $\Delta w_{ij} = \eta \cdot s_i \cdot s_j$, where η is the learning rate and s_i, s_j are the pre- and post-synaptic activities. *Status: theoretical. No implementation or experimental validation.*
2. **Homeostatic plasticity.** Regulatory mechanisms that maintain neural activity within functional ranges, preventing runaway excitation or silence. Each node adjusts its excitability to maintain a target firing rate. *Status: theoretical. No implementation.*

3. **Structural plasticity.** Long-term topology changes, including the formation of new connections and the elimination of unused ones. Over time, the substrate's connectivity adapts to the statistics of its inputs. *Status: theoretical. No implementation.*

4. **Neuromodulatory plasticity.** Global signals that modulate neuronal gain based on task identity. Each node's activation is scaled by $\alpha_i(\mathbf{n}_t) = \mathbf{r}_i \cdot \mathbf{n}_t$, where $\mathbf{r}_i$ is the evolved receptor density and $\mathbf{n}_t$ is the task's neurotransmitter vector. *Status: plausible future direction* (§8.4).

Of the four proposed plasticity mechanisms, all remain future directions; each is a design aspiration rather than an implemented capability. Whether Hebbian, homeostatic, or structural plasticity would provide additional value beyond neuromodulatory gating, or whether neuromodulatory gating alone suffices for multi-task operation, is an open question.

B.7 GEENNS among Continual Learning Approaches

Note: GEENNS has not been tested on continual learning tasks. This table positions the design intent, not validated capabilities.

Catastrophic forgetting (the tendency of neural networks to lose previously learned information when trained on new tasks) has been recognized as a fundamental challenge since the early connectionist literature [31, 65]. To understand what GEENNS contributes to this problem, it helps to position it among existing approaches to continual and multi-task learning. Table B.4 compares GEENNS against established methods along five dimensions.

Table B.4: GEENNS compared with established continual learning and modular approaches. "Encoding" distinguishes direct (one gene per weight) from indirect (CPPN-generated). "Forgetting prevention" describes the primary mechanism for retaining prior task performance.

Method	**Topology**	**Encoding**	**Multi-task**	**Forgetting prevention**
EWC [51]	Fixed	Direct	Sequential	Weight regularization
PackNet [62]	Fixed	Direct	Sequential	Weight freezing by mask
PNN [80]	Growing	Direct	Sequential	Lateral connections
MoE [43]	Fixed gated	Direct	Simultaneous	Gated routing
Modular NE	Evolved modules	Direct	Separate	Separate populations
GEENNS	**Evolved adaptive**	**Indirect**	**Compositional**	**Frozen grids + evolved mappers**

Three properties distinguish GEENNS from all entries in the table:

1. **Indirect encoding with evolved adaptive topology.** GEENNS substrates are generated by CPPNs through EMR-HyperNEAT, enabling compact genomes to produce large-scale structured networks with the potential for per-node function diversity. No other method in the table uses indirect encoding.

2. **Compositional multi-task operation.** Rather than learning tasks sequentially (EWC, PackNet, PNN) or training all tasks simultaneously (MoE), GEENNS trains specialist grids independently and then composes them through evolved mappers. Tasks are decomposed and delegated, not sequentially accumulated.

3. **Forgetting prevention through architectural separation.** Grids are frozen after specialist training. New tasks cannot overwrite grid weights because mappers, not grids, are evolved for each new task. This is structurally different from regularization (EWC), masking (PackNet), or growing (PNN).

The key caveat: this architectural separation has been validated up to 5-grid Boolean scale (with 6-grid recoverable via gated routing). Whether it scales to dozens of grids and non-trivial tasks, and whether the mapper complexity remains tractable, is unknown. The comparison table positions GEENNS among these methods; it does not claim superiority.

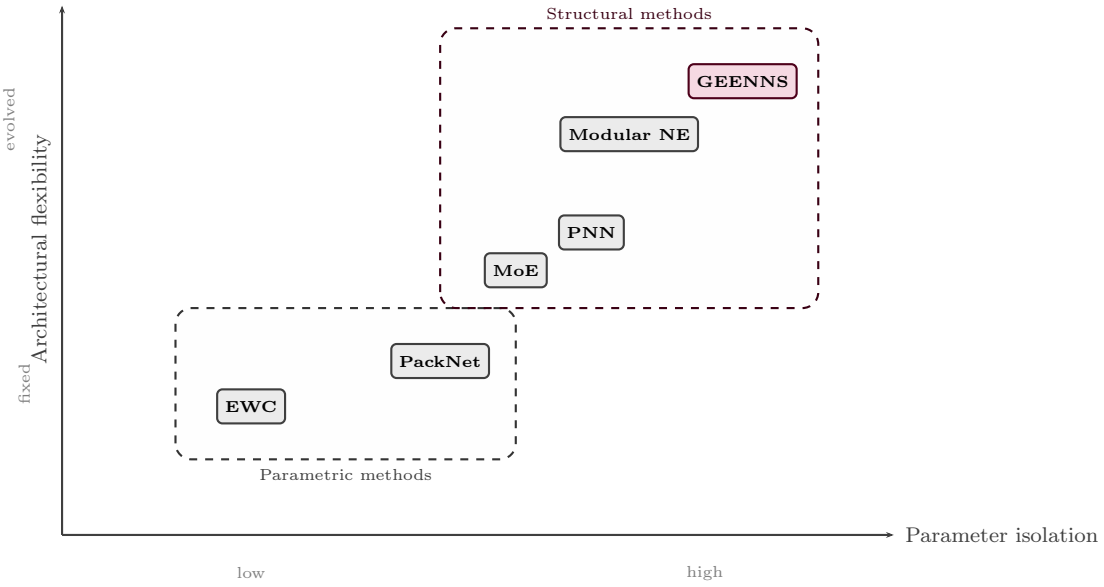

Figure B.7: GEENNS positioned among continual learning methods. The x-axis measures degree of parameter isolation (from shared weights to fully separate components); the y-axis measures architectural flexibility (from fixed topology to fully evolved). GEENNS occupies the upper-right: evolved adaptive topology with full parameter isolation through frozen specialist grids.

B.8 Formal Mathematical Specification

We present the mathematical formalization of GEENNS's core components. The formalism below captures the current design intent: a specification target, not a description

of implemented behavior. None of the definitions, dynamics, or fitness functions below have been realized in code beyond the simplified prototype of Section 7.6. This section provides the specification at a level sufficient for implementation and for connecting to the experimental contributions in subsequent chapters.

B.8.1 Grid Formalism

Definition B.1 (Grid)**.** *A GEENNS grid is a tuple*

$$\mathcal{G} = (V, E, f, g, \mathbf{R})$$

where:

- $V = \{v_1, \ldots, v_n\}$ *is a set of nodes, each with spatial coordinates* $(c_i, l_i, n_i) \in \mathbb{N}^3$ *where* c_i *is the column index,* l_i *is the layer index, and* n_i *is the within-layer node index.*
- $E \subseteq V \times V \times \mathbb{R}$ *is a set of weighted directed connections, where* $(v_i, v_j, w_{ij}) \in E$ *indicates a connection from* v_i *to* v_j *with weight* $w_{ij} \in \mathbb{R}$*.*
- $f : V \to \mathcal{F}$ *assigns each node an activation function from the palette* $\mathcal{F} = \{f_1, \ldots, f_p\}$ *(e.g.,* $\{\texttt{tanh}, \texttt{sin}, \texttt{sigmoid}, \texttt{relu}, \ldots\}$*).*
- $g : V \to \mathcal{A}$ *assigns each node an aggregation function from the set* $\mathcal{A} = \{g_1, \ldots, g_q\}$ *(e.g.,* $\{\texttt{sum}, \texttt{min}, \texttt{max}, \texttt{mean}\}$*).*
- $\mathbf{R} \in \mathbb{R}^{|V| \times 4}$ *is the receptor density matrix, where row* $\mathbf{r}_i = [r_i^{DA}, r_i^{5HT}, r_i^{NE}, r_i^{ACh}]$ *specifies the sensitivity of node* v_i *to each neurotransmitter.*

The spatial coordinates enforce the columnar organization: nodes with the same column index c_i belong to the same column, and within a column, nodes are organized into layers by l_i. The canonical microcircuit template is captured by the constraint that the intra-columnar connectivity pattern $\{(v_i, v_j, w_{ij}) \in E \mid c_i = c_j\}$ is identical (up to weight values) across all columns.

B.8.2 State Dynamics

Given a grid $\mathcal{G}$, the state vector $\mathbf{s}(t) = [s_1(t), \ldots, s_{|V|}(t)]^T \in \mathbb{R}^{|V|}$ evolves under discrete-time dynamics. For a node v_i with activation function $f(v_i)$, aggregation function $g(v_i)$, and incoming connections from nodes $\{v_j : (v_j, v_i, w_{ji}) \in E\}$:

$$s_i(t+1) = f(v_i)(g(v_i)(\{w_{ji} \cdot s_j(t) : (v_j, v_i, w_{ji}) \in E\}) + b_i) \tag{B.1}$$

where b_i is the bias of node v_i. Under a neuromodulatory extension with task-specific neurotransmitter (NT) vector $\mathbf{n}_t \in \mathbb{R}^4$, the update becomes:

$$s_i(t+1) = f(v_i)(\alpha_i(\mathbf{n}_t) \cdot g(v_i)(\{w_{ji} \cdot s_j(t)\}) + b_i) \tag{B.2}$$

where the gain modulation factor $\alpha_i(\mathbf{n}_t) = \mathbf{r}_i \cdot \mathbf{n}_t$ is the dot product of node v_i's receptor density with the task's NT vector.

B.8.3 Mapper Formalism

Definition B.2 (Mapper). *A mapper for task t over grids $\mathcal{G}_1, \ldots, \mathcal{G}_k$ is a tuple*

$$\mathcal{M}_t = (M_{in}, M_{out}, M_{inter})$$

where:

- $M_{in} : \mathbb{R}^{d_{in}} \to \prod_{i=1}^{k} \mathbb{R}^{|V_i^{input}|}$ *is the input mapper that decomposes the task input* $\mathbf{x} \in \mathbb{R}^{d_{in}}$ *into injection signals for each grid's input nodes.*
- $M_{out} : \prod_{i=1}^{k} \mathbb{R}^{|V_i^{output}|} \to \mathbb{R}^{d_{out}}$ *is the output mapper that composes the grids' output signals into the task output* $\mathbf{y} \in \mathbb{R}^{d_{out}}$.
- $M_{inter} : \prod_{i=1}^{k} \mathbb{R}^{|V_i|} \to \prod_{i=1}^{k} \mathbb{R}^{|V_i|}$ *is the inter-grid mapper that routes signals between grids during processing.*

Each component of $\mathcal{M}_t$ is implemented as a NEAT-evolved network, allowing the mapper's topology and weights to evolve against frozen grid architectures. The compositional forward pass for a task with input $\mathbf{x}$ is:

$$\begin{aligned}
\mathbf{x}_1, \ldots, \mathbf{x}_k &= M_{\text{in}}(\mathbf{x}) && \text{(decompose)} && \text{(B.3)}\\
\mathbf{s}_i^{(0)} &= \text{inject}(\mathbf{x}_i, \mathcal{G}_i) && \text{(inject)} && \text{(B.4)}\\
\mathbf{s}_i^{(t+1)} &= F_{\mathcal{G}_i}(\mathbf{s}_i^{(t)}, M_{\text{inter}}(\mathbf{s}_1^{(t)}, \ldots, \mathbf{s}_k^{(t)})) && \text{(propagate)} && \text{(B.5)}\\
\mathbf{y} &= M_{\text{out}}(\text{extract}(\mathbf{s}_1^{(T)}), \ldots, \text{extract}(\mathbf{s}_k^{(T)})) && \text{(compose)} && \text{(B.6)}
\end{aligned}$$

where $F_{\mathcal{G}_i}$ is the state update function of grid $\mathcal{G}_i$ (Equation B.1 or B.2), T is the number of propagation steps, and inject/extract select the appropriate input/output nodes.

B.8.4 CPPN Connectivity Generation

Connection existence and weights are generated by CPPNs queried at spatial coordinates. For a connection between source node v_i at normalized coordinates $(\hat{c}_i, \hat{l}_i)$ and target node v_j at $(\hat{c}_j, \hat{l}_j)$:

$$(w_{ij}, p_{ij}) = \mathcal{C}(\hat{c}_i, \hat{l}_i, \hat{c}_j, \hat{l}_j) \tag{B.7}$$

where $\mathcal{C} : \mathbb{R}^4 \to \mathbb{R}^2$ is the CPPN, w_{ij} is the connection weight, and p_{ij} is the connection existence probability. A connection exists if $|p_{ij}| > \theta_{\text{conn}}$, where θ_{conn} is the connection existence threshold (Chapter 2).

For multi-output CPPNs that could also assign per-node functions:

$$(w_{ij}, p_{ij}, f_j, g_j) = \mathcal{C}_{\text{ext}}(\hat{c}_i, \hat{l}_i, \hat{c}_j, \hat{l}_j) \tag{B.8}$$

where $f_j \in \{1, \ldots, p\}$ is the activation function index and $g_j \in \{1, \ldots, q\}$ is the aggregation function index assigned to the target node.

B.8.5 Development Function

The genome-to-phenotype transformation $\mathcal{D}$ maps a genome Γ to a functional grid:

$$\mathcal{D} : \Gamma \to \mathcal{G} \tag{B.9}$$

where $\Gamma = (\Gamma_{\text{NEAT}}, \Gamma_{\text{CPPN}}, \Gamma_{\text{micro}})$ consists of the NEAT genome for the canonical microcircuit (Γ_{micro}) and the CPPN genomes for connectivity patterns (Γ_{CPPN}), both encoded within the NEAT framework (Γ_{NEAT}). The development function proceeds through the five stages described in Section B.5 and illustrated in Figure B.6.

This function is deterministic: $\mathcal{D}(\Gamma)$ always produces the same grid for the same genome.

B.8.6 Multi-Task Fitness

For tasks $\{t_1, \ldots, t_T\}$ with mappers $\{\mathcal{M}_{t_1}, \ldots, \mathcal{M}_{t_T}\}$, the fitness of a genome Γ is:

$$\phi(\Gamma) = \min_{t \in \{t_1, \ldots, t_T\}} \phi_t(\mathcal{D}(\Gamma), \mathcal{M}_t) \tag{B.10}$$

where $\phi_t(\mathcal{G}, \mathcal{M}_t)$ is the task-specific fitness of grid $\mathcal{G}$ using mapper $\mathcal{M}_t$. The min aggregation ensures that the genome must perform well on *all* tasks to achieve high fitness, preventing specialization on a subset. Convergence is defined as $\phi(\Gamma) \geq \phi^* = 0.95$.

This fitness function would have a direct analog under neuromodulation, where a task-specific NT vector $\mathbf{n}_t$ would serve the role of the mapper $\mathcal{M}_t$: different NT vectors would route the same topology through different effective configurations, and fitness would be the minimum across all task-specific evaluations.